



Govt. Post Graduate College Civil Lines Sheikhupura

Semester Project Report

College Academic system Database

Submitted by

Muhammad Muzzamal 110837

Imran Ali 110855

Department of Information Technology

Govt. Post Graduate College Civil Lines Sheikhupura

2025

CERTIFICATE

This is to certify that M. Muzzamal and Imran Ali bearing Registration No. 2023-KS-149 and 2023-KS-148 has completed the semester final project titled as “College Academic System” at the Department of Information Technology, Govt. Post Graduate College Civil Lines Sheikhpura.

Supervisor

Mam Sehrish Khan

Signature: _____

Internal Panel

Member 1: _____

Signature: _____

Member 2: _____

Signature: _____

DECLARATION

We declare that the work in this project titled “College Academic System” was done completely by us under the guidance of **Madam Sahrish Khan** from Information Technology, Govt. Post Graduate Civil Lines Sheikhpura.

We confirm that:

1. This project is our own work and study.
2. Everything in this project was done by us.
3. We didn't copy from anyone else.
4. I have given proper credit to all sources I used.

I know that if I broke any college rules, I may face punishment.

Name : Muhammad Muzzamal

Registration No. : 2023-KS-149

Name : Imran Ali

Registration No. : 2023-KS-148

ACKNOWLEDGEMENT

I really feel extremely privileged to take this chance to precise my heartiest gratitude and deep sense of indept to Head of institute in regard to his kindness and scholastic guidance, eager, couriosity and constant encouragement.

We are thnakful to ALLAH and our respected teacher becuese our teacher's efforts are so important for us. We were not able to perform this task and we can't complete this project without their hard work. Thank to our family and our teachers for their trust and support.

My special and heartily thanks to my supervisor, **Madam Sehrish Khan**, who encouraged and directed us. Her challenges brought this work towards a completion. It's because of her we achieve this task, and this project came into exitance.

Abstract

The College Academic System Database is a structured and efficient database system designed to streamline academic operations within a college environment. This project aims to digitize and manage essential academic data such as student records, course enrollments, attendance, marks, and faculty details through a centralized database.

The primary objective of this project is to enhance the accuracy, security, and accessibility of academic information. By replacing traditional paper-based methods with a database-driven solution, the system reduces human error, improves data integrity, and allows for faster data retrieval and reporting.

The system is developed using relational database concepts and implements structured query language (SQL) for handling data operations. It is designed to be scalable and adaptable to various academic departments and administrative needs.

This project not only provides hands-on experience with database design and implementation but also demonstrates the importance of data management in educational institutions.

TABLE OF CONTENTS

CHAPTER NO 1	INTRODUCTION
• Introduction • Statement Problem • Objectives • Scope of project	
CHAPTER NO 2	LITERATURE REVIEW
• Literature View • Previous Work • Key Observation	
CHAPTER NO 3	SYSTEM ANALYSIS
• System requirement • Existing System Overview • Proposed System: • Functional Requirements: • Non-Functional Requirements	
CHAPTER NO 4	SYSTEM DESIGN
• System Design • System Architecture • Database Design • Entity-Relationship Diagram (ERD) • UML Diagrams • Entity Relation Diagram using Chen Model: • Entity Relation Diagram using Crow Feet Model:	
CHAPTER NO 5	NORMALIZATION
• Normalization: • Normal Form: • Normalization of Tables:	
CHAPTER NO 6	INTRODUCTION to SQL:
• Introduction to SQL: • Purpose and Importance • Key Functions of SQL • SQL DATA MANIPULATION COMMANDS • SQL DATA DEFINITION COMMANDS	

CHAPTER NO 7

SQL QUERIES

- SQL Queries:
- Order of SQL Query Clauses
- Create Statement:
- SELECT Statement:
- INSERT Statement:
- UPDATE Statement:

CHAPTER NO 8

JOIN

- Join:
- Types of joins:

CHAPTER NO 9

SQL SERVER DATABASE CREATION

- Introduction:
- DATABASE creation:
- • CREATE TABLES:
- Add FOREIGN KEYS (RELATION)
- Description

Introduction

In today's digital age, educational institutions are increasingly adopting database systems to manage their academic and administrative tasks efficiently. Manual handling of student records, attendance, course registrations, and faculty data often leads to inefficiencies, errors, and data loss. To overcome these challenges, there is a strong need for a centralized and structured system that ensures accurate record-keeping and smooth information flow. The College Academic System Database project is designed to automate and simplify academic data management within a college environment. This system stores vital academic information such as student profiles, teacher records, course details, marks, and attendance in a well-organized and easily retrievable format.

Problem Statement:

“College Academic System with Results and Teaching Load: This system handles student enrollment, course registration, attendance, assessments, and grade processing. Professors have maximum teaching loads, and class timetables are auto-generated to avoid conflicts. Students have access to academic history, backlogs, and improvement options. Faculty members submit grades online and can mark attendance via an internal app. The system tracks results and generates department-wise performance analytics.”

Objectives

- To design and implement a relational database system for academic data.
- To reduce manual workload and human errors in record keeping.
- To provide secure, accurate, and easily accessible academic records.
- To enable efficient reporting and data analysis for decision-making.

Scope of the Project:

This project focuses on developing a backend database system for academic purposes. It includes modules for student information, faculty records, course registration, and academic performance. The current version is designed for small to medium-sized educational institutions and can be extended in the future with a user interface and web-based access.

Literature Review

The development of academic management systems has been a subject of continuous research and innovation over the past few decades. Many institutions have transitioned from traditional paper-based record systems to computerized database systems to increase efficiency, accuracy, and accessibility.

Various studies and existing systems highlight the benefits of digital academic databases. These systems enable easy access to student data, course information, attendance tracking, and academic performance analysis. Moreover, database systems offer improved data integrity and security compared to manual systems.

Previous Work:

Several academic management systems have been developed using different technologies, including:

- **Oracle-based Academic Management Systems** used in large universities for storing and retrieving complex student data across multiple departments.
- **Microsoft Access and Excel-based Solutions** used in small colleges, which, although easy to use, lack scalability and security.
- **Open-source Solutions like Moodle**, which combine database functionality with learning management, although more focused on e-learning than core data handling.

Key Observations

- Most successful systems are built on relational databases using SQL.
- Integration of secure login features and role-based access control significantly improves data privacy.

This project builds on these insights by focusing on a custom-designed relational database specifically tailored for college academic environments, ensuring efficiency, accuracy, and future scalability.

System Analysis / Requirements:

This section outlines the analysis of the existing manual system, identifies its limitations, and defines the functional and non-functional requirements for the **College Academic System Database**.

Existing System Overview

Currently, many academic institutions still rely on manual or semi-digital methods for managing student and faculty records. These include physical files, spreadsheets, or non-centralized software. Such methods are:

- Time-consuming
- Prone to errors and data loss
- Difficult to update or search
- Not secure or scalable

Proposed System:

The proposed system is a relational database designed to store and manage all essential academic records digitally. It offers fast data access, better organization, and enhanced security. The system supports operations like inserting, updating, deleting, and retrieving academic data.

Functional Requirements:

1. Student Management

- Add, update, delete, and view student records
- Assign students to specific courses and semesters

2. Faculty Management

- Maintain faculty profiles
- Assign subjects and classes

3. Course Management

- Add and manage course details
- Allocate faculty to courses

4. Attendance and Marks Recording

- Record and view student attendance
- Enter and manage marks and results

5. Report Generation

- Generate academic reports based on stored data

Non-Functional Requirements

- **Performance:** System should respond to queries within seconds.
- **Security:** Only authorized personnel can access and modify data.
- **Scalability:** Should allow future expansion for new modules or departments.
- **Maintainability:** Easy to update and maintain database structure.

System Design

The College Academic System Database is designed using a relational database management system (RDBMS). The system is modular in structure, with different components responsible for managing student data, faculty records, course management, attendance, and marks. This section includes an overview of the system architecture, database design, and important diagrams.

System Architecture

The system is built with a three-tier architecture:

1. Database Layer:

This is the foundation of the system, where all academic data is stored in tables using a Relational Database Management System (RDBMS). We use SQL to manage data operations such as insert, update, and query.

2. Business Logic Layer:

The business logic layer contains all the rules for how the data is processed and manipulated. It will handle operations like calculating marks, generating reports, and handling user authentication and authorization.

3. Presentation Layer (Frontend – Optional in Future):

This is where the user interacts with the system. Initially, it will be a backend-only system, but in the future, it can be extended to include a web interface or desktop application.

Database Design

The database design consists of several **tables** to store the different types of information related to students, faculty, courses, and attendance. The following **ER diagram** outlines the relationships between key entities:

- **Students Table:** Stores personal details, course enrollments, and attendance data.
- **Faculty Table:** Stores faculty details, assigned courses, and their respective schedules.
- **Courses Table:** Stores course information such as course code, title, and faculty assignment.
- **Attendance Table:** Tracks attendance for each student in each class.
- **Marks Table:** Records marks for each student in each subject.
- **Timetable table:** This helps to calculate the teaching load.

- **Department table:** This helps to store department information like deptNo and deptName.
- **Assessment table:** This stores the academic records of student and their grades.

Entity-Relationship Diagram (ERD)

Below is the structure of the key entities and relationships:

1. **Student (StudentID, Name, DOB, Address, Email)**
2. **Faculty (FacultyID, Name, Department, Email)**
3. **Course (CourseID, CourseName, FacultyID)**
4. **Attendance (AttendanceID, StudentID, CourseID, Date, Status)**
5. **Marks (MarksID, StudentID, CourseID, MarksObtained)**
6. **Department (deptID, deptName)**
7. **Enrollment (EnrolID, Semester, Year, Grade)**
8. **Assessment (AssID, AssType, Date, Marks)**

The relationships include:

- A student can enroll in multiple courses.
- A faculty member can teach multiple courses.
- Attendance and marks are recorded for each student per course.

UML Diagrams

1. Use Case Diagram

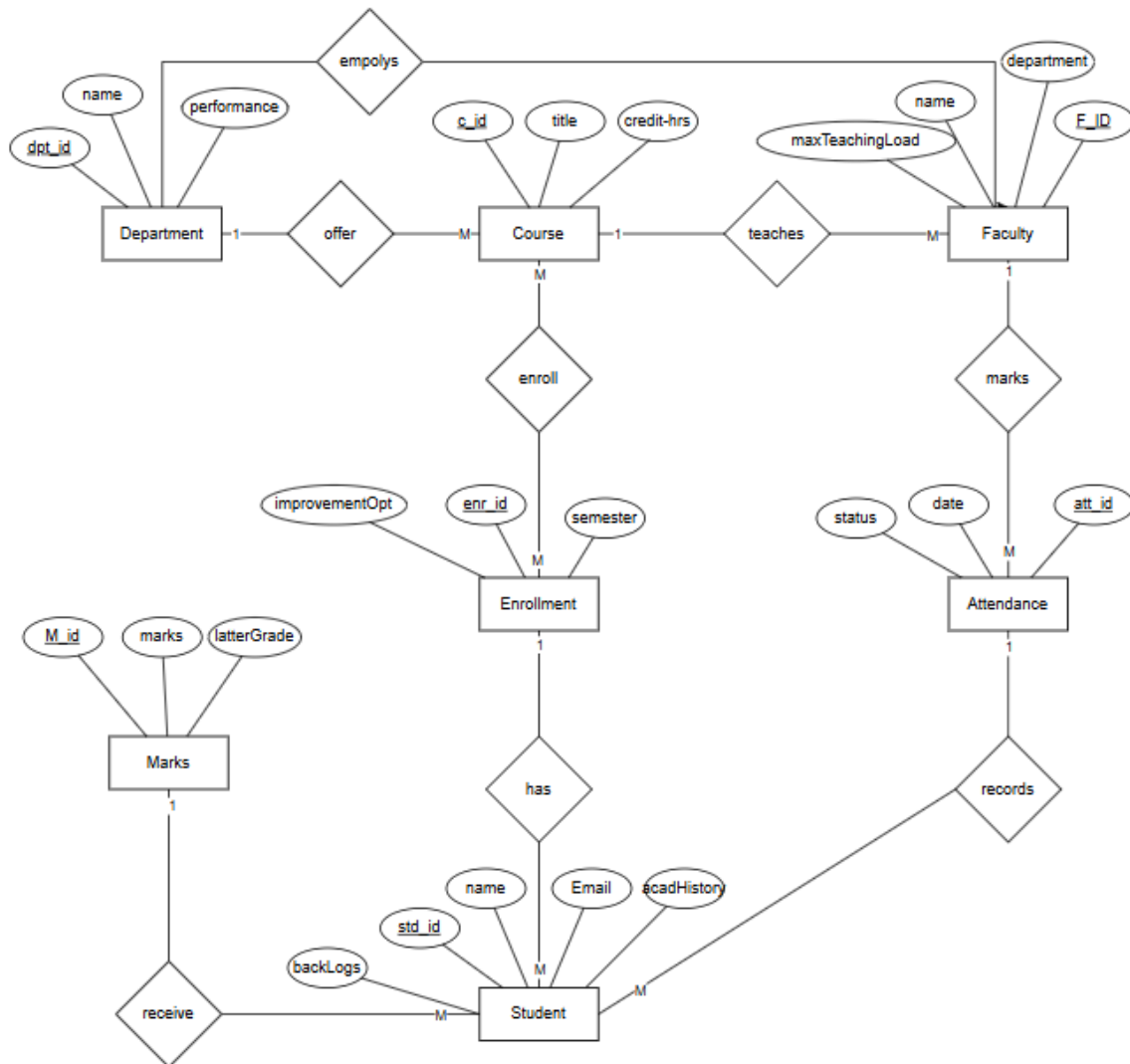
A use case diagram will be designed to show the interactions between the user (administrator, faculty, students) and the system for key functionalities such as adding student records, marking attendance, and generating reports.

2. Class Diagram

A class diagram would define the objects within the system, their properties, methods, and relationships.

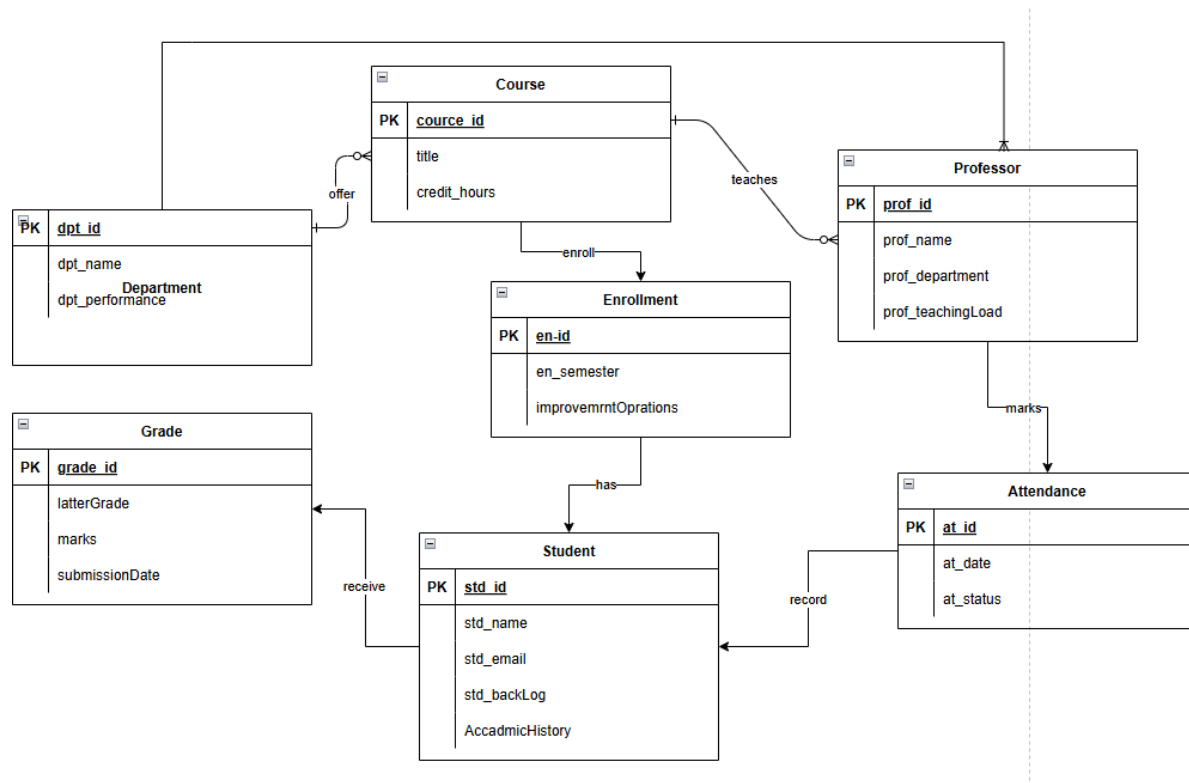
Entity Relation Diagram using Chen Model:

The Chen Model is the original Entity-Relationship (ER) approach, using rectangles for entities, ovals for attributes, and diamonds for relationships. It emphasizes conceptual clarity with explicit cardinality notation (1:N, M:N). Though less common in modern tools, it remains foundational for database design theory. Ideal for early-stage modeling and academic use.



Entity Relation Diagram using Crow Feet Model:

The Crow's Foot notation is a modern ERD standard that uses symbols (|, o, >) to represent cardinality (1:1, 1:N, M:N). Entities appear as boxes, relationships as labeled lines, with "crow's feet" (⌋) indicating the "many" side. It's widely used in industry tools like Lucidchart and MySQL Workbench for its intuitive visual design. More compact than Chen notation but equally powerful for database modeling.



Normalization:

“Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. It involves decomposing tables into smaller, related tables and defining relationships between them.”

Objective Of Normalization:

- **Minimize Redundancy:** Avoid duplicate data to save storage space.
- **Improve data Integrity:** Ensure accuracy and consistency by reducing anomalies (insertion, update, deletion) .
- **Simple Database Structure:** Make the database easier to maintain and query.

Normal Form:

1NF (First Normal Form):

Rule of thumb:

“Ensure each column holds atomic values (no multiples), eliminate repeating groups, and use unique column names.”

2NF (2nd Normal Form):

Rule of thumb:

- *Must be in 1NF.*
- *No partial dependency – All non-key attributes must depend on the entire primary key (not just part of it).*

3NF (3rd Normal Form):

Rule:

- *Must be in 2NF.*
- *No transitive dependency (non-key attributes should not depend on other non-key attributes).*

Normalization of Tables:

Unnormaization form:

Std ID	StdName	Dept Name	Faculty Name	ProfessorName	Course Name	EnrollmentDate	AttendanceDate	AssignmentTitle	Marks	Grade
--------	---------	-----------	--------------	---------------	-------------	----------------	----------------	-----------------	-------	-------

These table have a lot of redundancy and anomalies.

1st Normal Form:

Remove repeating groups and ensure atomicity

- Each cell should hold atomic (single) values.
- Separate repeating attributes (like multiple courses per student) into multiple rows.

Split big table into multiple tables, e.g.,

- Student(StudentID, StudentName, DOB, Email, Address)
- Faculty(FacultyID, FacultyName)
- Department(DepartmentID, DepartmentName, FacultyID)
- Professor(ProfessorID, ProfessorName, DepartmentID)
- Course(CourseID, CourseName, DepartmentID, ProfessorID)
- Enrollment(EnrollmentID, StudentID, CourseID, EnrollmentDate)
- Attendance(AttendanceID, EnrollmentID, Date, Status)
- Assignment(AssignmentID, CourseID, Title, DueDate)
- Marks(MarksID, EnrollmentID, AssignmentID, MarksObtained)

2nd Normal Form:

Why is 2NF important?

2NF removes partial dependencies, which occur when a non-key attribute depends on only part of a composite key, leading to data redundancy and update anomalies.

Remove partial dependencies (apply when composite keys exist)

Look for tables with composite keys (like Enrollment which has StudentID + CourseID):

- **Enrollment** has composite key (StudentID, CourseID).
- Make EnrollmentID a surrogate PK or keep composite PK.
- Move attributes depending only on part of the key to separate tables.

Since EnrollmentDate depends on full key (StudentID + CourseID), it stays. No partial dependencies here.

- Student(StudentID, StudentName, DOB, Email, Address)
- Faculty(FacultyID, FacultyName)
- Department(DepartmentID, DepartmentName, FacultyID)
- Professor(ProfessorID, ProfessorName, DepartmentID)
- Course(CourseID, CourseName, DepartmentID, ProfessorID)
- Enrollment(EnrollmentID, StudentID, CourseID, EnrollmentDate)
- Attendance(AttendanceID, EnrollmentID, Date, Status)
- Assignment(AssignmentID, CourseID, Title, DueDate)
- Marks(MarksID, EnrollmentID, AssignmentID, MarksObtained)

3rd Normal Form:

Importance of 3NF:

3NF eliminates transitive dependencies, which improves data integrity and reduces redundancy. In this form, every non-key attribute must depend only on the primary key and not on another non-key attribute.

- In **Department**, FacultyName depends on FacultyID → FacultyName should be in Faculty table only. Department only stores FacultyID.
- In **Professor**, DepartmentName should NOT be stored. Only DepartmentID is stored, department info is accessed via join.

Make sure no column depends on non-key attributes.

❖ Student

- **Primary Key:** StudentID
- **Foreign Keys:** —

❖ Faculty

- **Primary Key:** FacultyID
- **Foreign Keys:** —

❖ Department

- Primary Key: DepartmentID
- Foreign Keys: FacultyID

❖ Professor

- Primary Key: ProfessorID
- Foreign Keys: DepartmentID

❖ Course

- Primary Key: CourseID
- Foreign Keys: DepartmentID, ProfessorID

❖ Enrollment

- Primary Key: EnrollmentID
- Foreign Keys: StudentID, CourseID

❖ Attendance

- Primary Key: AttendanceID
- Foreign Keys: EnrollmentID

❖ Assignment

- Primary Key: AssignmentID
- Foreign Keys: CourseID

❖ Marks

- Primary Key: MarksID
- Foreign Keys: EnrollmentID, AssignmentID

Introduction to SQL:

Definition:

Structured Query Language (SQL) is a standard programming language used to manage and manipulate relational databases. It allows users to perform various operations such as creating, retrieving, updating, and deleting data within a database.

Purpose and Importance

The main purpose of SQL is to efficiently organize data and retrieve meaningful information by running queries. SQL is human-readable and uses simple commands that make it easy to interact with databases. These commands include:

- **SELECT** — to retrieve data
- **INSERT** — to add new data
- **UPDATE** — to modify existing data
- **DELETE** — to remove data

Key Functions of SQL

SQL's functions fall into several broad categories:

1. **Data Manipulation Language (DML):**
Commands that allow insertion, updating, deletion, and retrieval of data within database tables. These commands enable users to interact dynamically with the data.
2. **Data Definition Language (DDL):**
Commands used to create database objects such as tables, indexes, and views, and to define access rights to these objects.
3. **Transaction Control Language (TCL):**
DML commands are executed within transactions — logical units of work defined by business rules. TCL commands control the processing of these transactions, ensuring data integrity.
4. **Data Control Language (DCL):**
Commands to control access permissions on data objects, such as granting or restricting user rights on tables.

SQL DATA MANIPULATION COMMANDS

Commands	Description
SELECT	Select attributes from rows in one or more tables or views.
FROM	Specifies the tables from which data should be retrieved
WHERE	Restricts the selection of rows based on a conditional expression
GROUP BY	Groups the selected rows based on one or more attributes
HAVING	Restricts the selection of grouped rows based on a condition
ORDERED BY	Orders the selected rows based on one or more attributes
INSERT	Inserts row(s) into a table
UPDATE	Modifies an attribute's values in one or more table's rows
DELETE	Deletes one or more rows from a table
Comparison Operators	
=, >, <=, >=, <, !=	Used in conditional expressions
Logical operators	
AND/OR/NOT	Used in conditional expressions
Special operators	
BETWEEN	Checks whether an attribute value is within a range
IN	Checks whether an attribute value matches any value within a value list
LIKE	Check whether an attribute value matches a given string pattern
IS NULL	Check whether an attribute value is NULL
EXISTS	Check whether a subquery return any rows
DISTINCT	Limits values to unique values
Aggregate Functions	
COUNT	Return the number of rows with not-null values for a given column
MIN	Return the minimum attribute value found in a given column
MAX	Return the maximum attribute value found in a given column
SUM	Return the sum of all values for a given column
AVG	Return the average of all values for a given column

SQL DATA DEFINITION COMMANDS

Commands	Description
CREATE SCHEMA AUTHORIZATION	Create a database schema
CREATE TABLE	Create a new table in the user's database schema
NOT NULL	Ensure that a column will not have null values
UNIQUE	Ensure that a column will not have duplicate
PRIMARY KEY	Define a primary key for a table
FOREIGN KEY	Define a foreign key for a table
DEFAULT	Defines a default value for a column (when no value is given)
CREATE INDEX	Create an index for a table
CREATE VIEW	Create a dynamic subset of row and column from one or more table
ALTER TABLE	Modifies a table's definition (adds, modifies, or deletes attribute or constraints)
CREATE TABLE AS	Create a new table based on a query in the user's database schema
DROP TABLE	Permanently deletes table (and its data)
DROP INDEX	Permanently delete an index
DROP VIEW	Permanently deletes a view

Commands	Description
Transection Control Language	
COMMIT	Permanently saves data changes
ROLLBACK	Restore data to its original values
Data Control Language	
GRANT	Gives a user permission to take a system action or access a data object
REVOKE	Remove a previously granted permission from a user.

SQL Queries:

In the SQL environment, the word query covers both **questions and actions**. Most SQL queries are used to answer questions such as these: *“What products currently held in inventory are priced over \$100, and what is the quantity on hand for each of those products?”* or *“How many employees have been hired since January 1, 2016, by each of the company’s departments?”*

Order of SQL Query Clauses

- **SELECT**—specifies the attributes to be returned by the query •
- **FROM**—specifies the table(s) from which the data will be retrieved
- **WHERE**—filters the rows of data based on provided criteria
- **GROUP BY**—groups the rows of data into collections based on sharing the same values in one or more attributes
- **HAVING**—filters the groups formed in the GROUP BY clause based on provided criteria
- **ORDER BY**—sorts the final query result rows in ascending or descending order based on the values of one or more attributes.

Create Statement:**Create new database:**

```
CREATE DATABASE college;  
  
CREATE DATABASE college IF NOT EXISTS;
```

Create table:

```
CREATE TABLE Students (StudentID INT PRIMARY KEY, Name  
VARCHAR(100), Age INT, Grade CHAR(1) );
```

CREATE INDEX

```
CREATE INDEX idx_name ON Students(Name);
```

CREATE VIEW

```
CREATE VIEW StudentNames AS SELECT Name FROM Students;
```

CREATE PROCEDURE

```
CREATE PROCEDURE GetAllStudents AS SELECT * FROM Students;
```

CREATE USER

```
CREATE USER "Muzzamal"@"localhost" IDENTIFIED BY '123456789';
```

SELECT Statement:

The SELECT query specifies the columns to be retrieved as a column list. The syntax for a basic SELECT query that retrieves data from a table is:

```
SELECT column-list FROM table-list;
```

Print all Records in a table

```
SELECT * FROM student;
```

Print Specific Columns

```
SELECT studentID, StudentName, Email FROM student;
```

Using Column Aliases

```
SELECT Student_name AS "Name", Student_Email AS "Email" FROM STUDENT;
```

Using Computed Columns

```
SELECT max_teaching_load*7 AS "Lectures in a week" FROM faculty;
```

Date Arithmetic (Date stored on the Days)

```
SELECT dob-12 FROM student;
```

Listing Unique Values:

```
SELECT DINCTINCT dept_name FROM department;
```

INSERT Statement:

This statement is used to enter records in the table.

Enter single record

```
INSERT INTO Students (StudentID, Name, Age, Grade) VALUES (1, 'Ali', 20, 'A');
```

Enter Multiple records:

```
INSERT INTO Students (StudentID, Name, Age, Grade) VALUES (1, 'Ali', 20, 'A'), (2, 'Ahmad', 21, 'B');
```


UPDATE Statement:

Update the existed record.

```
UPDATE Students SET student_name = 'Ahmad' WHERE StudentID = 2;
```

DELETE Statement:

Delete the specific record;

```
DELETE FROM student where student_id = 1;
```

DROP:

Permanently delete the table or database;

```
DROP TABLE student;
```

```
DROP industry;           //delete the database
```

Delete Statement:

Change the Structure of the table like add or delete the column.

```
ALTER TABLE Students ADD Email VARCHAR(100);
```

TRUNCATE Statement;

Delete the All-existing records of a tables but structure remain same.

```
TRUNCATE TABLE Students;
```

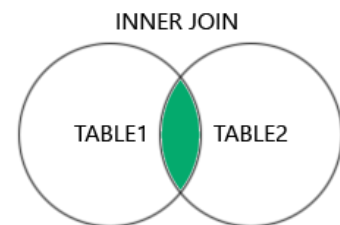
Join:

*“A **JOIN** is a SQL operation that allows you to combine rows from two or more tables based on a related column between them. When you want to retrieve related data stored in multiple tables and display it together as a single result, you use a **JOIN**.”*

Types of joins:**Inner join:**

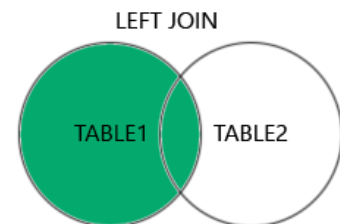
Returns only the rows that have matching values in both tables.

```
SELECT * FROM Table1 INNER JOIN Table2 ON  
Table1.t1_id = Table2.t1_id;
```

**LEFT JOIN (or LEFT OUTER JOIN)**

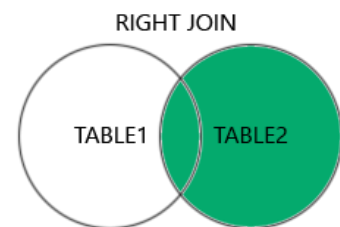
Returns all rows from the left table, and matched rows from the right table. If no match, shows NULL for right table columns.

```
SELECT * FROM Table1 LEFT JOIN Table2 ON  
Table1.t1_id = Table2.t1_id;
```

**Right JOIN (or RIGHT OUTER JOIN)**

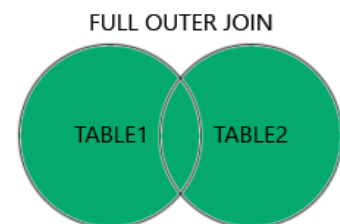
Returns all rows from the right table, and matched rows from the left table. If no match, shows NULL for left table columns.

```
SELECT * FROM Table1 RIGHT JOIN Table2 ON  
Table1.t1_id = Table2.t1_id;
```

**FULL JOIN (or FULL OUTER JOIN)**

Returns all rows when there is a match in one of the tables. Rows without match will have NULL in columns of the other table.

```
SELECT * FROM Table1 FULL JOIN Table2 ON  
Table1.t1_id = Table2.t1_id;
```



Introduction:

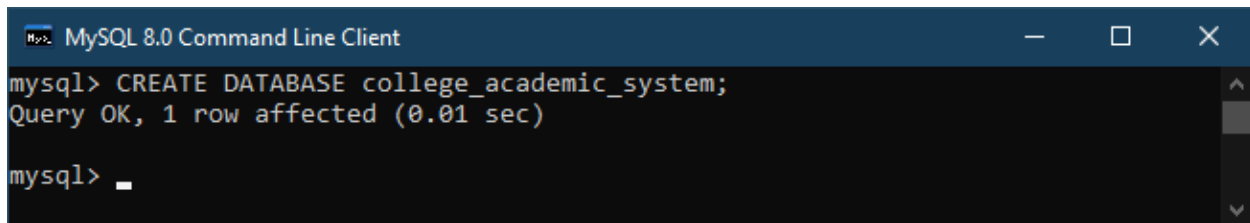
SQL Server is a powerful relational database management system developed by Microsoft. It allows users to create, manage, and query databases efficiently. This chapter will guide you through the process of creating your own database on SQL Server using SQL Server Management Studio (SSMS).

DATABASE creation:

- **Open the MYSQL Command Line Client**
- **Enter password**
- **Create database:**

Create a database in MYSQL Command Lines Client.

Command: CREATE DATABASE college_academic_system;



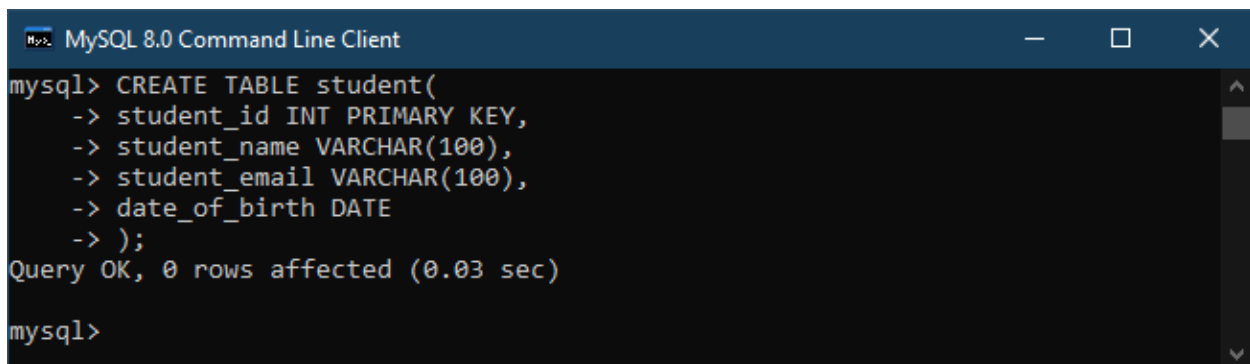
```
MySQL 8.0 Command Line Client
mysql> CREATE DATABASE college_academic_system;
Query OK, 1 row affected (0.01 sec)

mysql> _
```

- **CREATE TABLES:**

Create student table:

Command: CREATE TABLE student(student_id INT PRIMARY KEY, student_name VARCHAR(100), student_email VARCHAR(100), date_of_birth DATE);

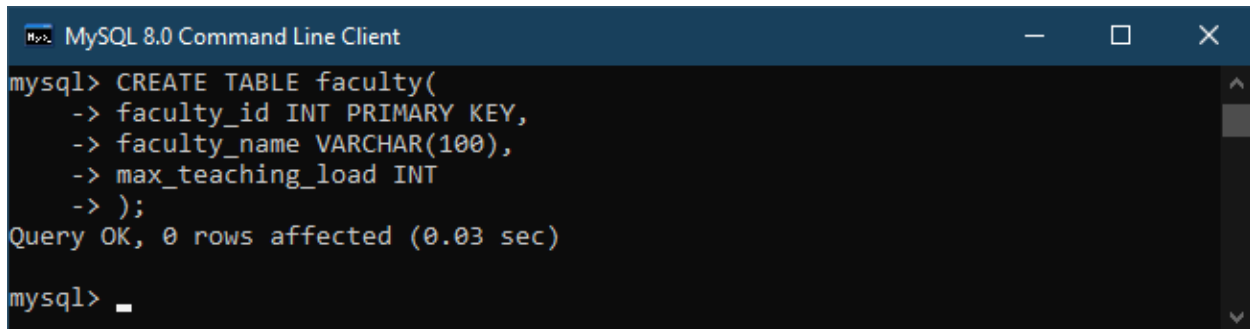


```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE student(
-> student_id INT PRIMARY KEY,
-> student_name VARCHAR(100),
-> student_email VARCHAR(100),
-> date_of_birth DATE
-> );
Query OK, 0 rows affected (0.03 sec)

mysql>
```

Create faculty table:

Command: `CREATE TABLE faculty(faculty_id INT PRIMARY KEY, faculty_name VARCHAR(100), max_teaching_load INT);`

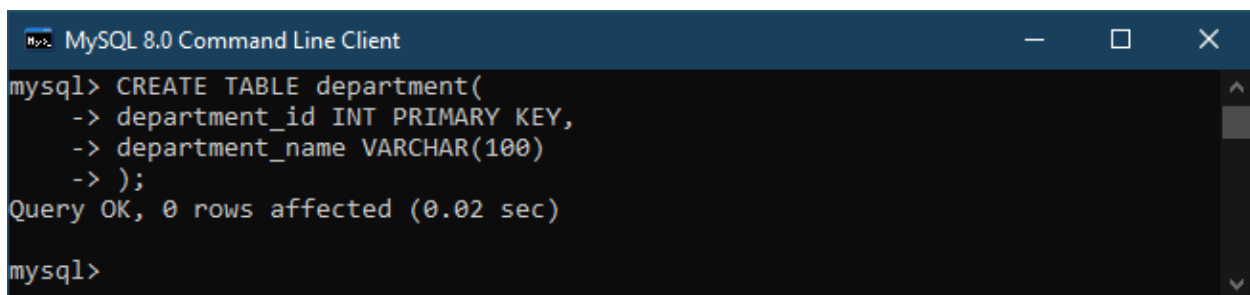


```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE faculty(
-> faculty_id INT PRIMARY KEY,
-> faculty_name VARCHAR(100),
-> max_teaching_load INT
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> _
```

Create department table:

Command: `CREATE TABLE department(department_id INT PRIMARY KEY, department_name VARCHAR(100));`

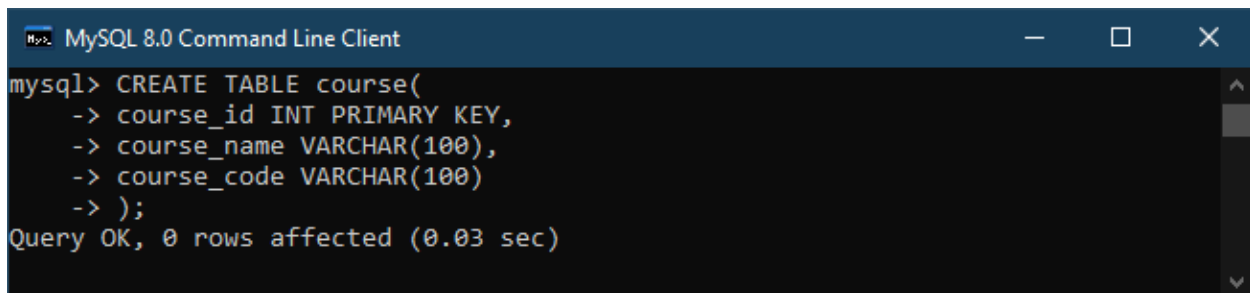


```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE department(
-> department_id INT PRIMARY KEY,
-> department_name VARCHAR(100)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql>
```

Create course table

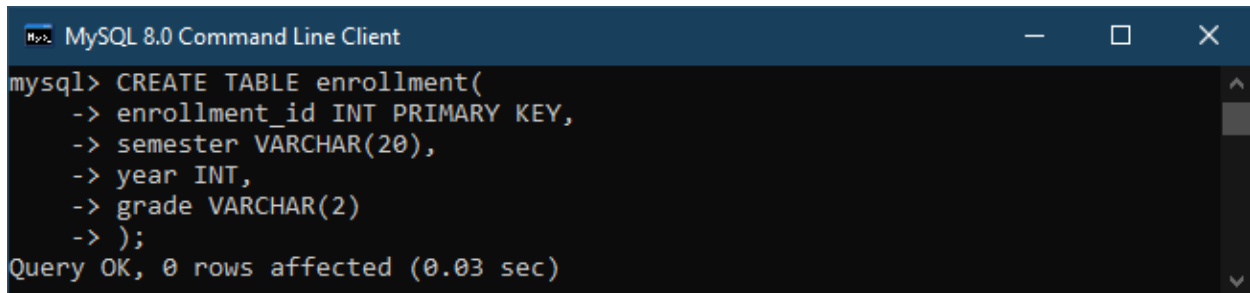
Command: `CREATE TABLE course(course_id INT, course_name VARCHAR(100), course_code VARCHAR(100));`



```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE course(
-> course_id INT PRIMARY KEY,
-> course_name VARCHAR(100),
-> course_code VARCHAR(100)
-> );
Query OK, 0 rows affected (0.03 sec)
```

Create enrollment table

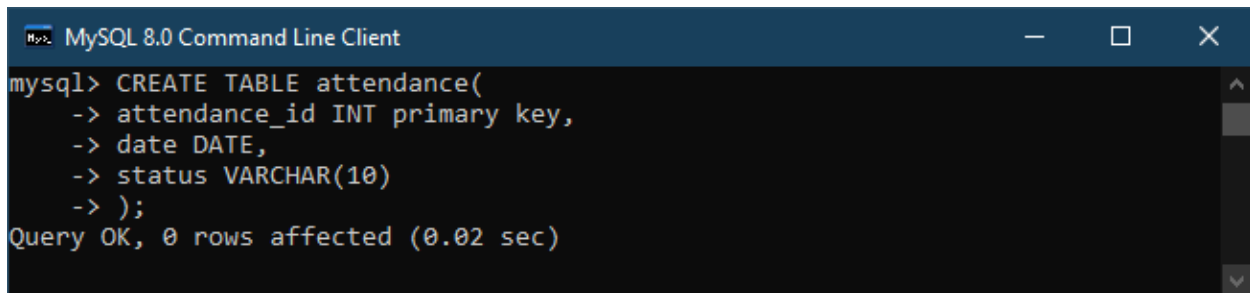
Command: `CREATE TABLE enrollment(enrollment_id INT PRIMARY KEY, semester VARCHAR(20), year INT, grade VARCHAR(2));`



```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE enrollment(
  -> enrollment_id INT PRIMARY KEY,
  -> semester VARCHAR(20),
  -> year INT,
  -> grade VARCHAR(2)
  -> );
Query OK, 0 rows affected (0.03 sec)
```

Create Attendance table:

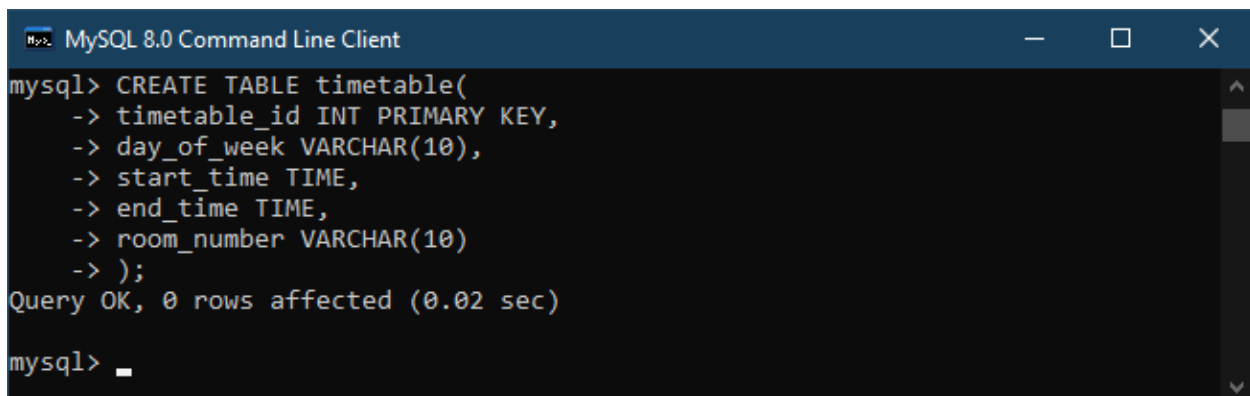
Command: `CREATE TABLE attendance(attendance_id INT PRIMARY KEY, date DATE, status VARCHAR(10));`



```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE attendance(
  -> attendance_id INT primary key,
  -> date DATE,
  -> status VARCHAR(10)
  -> );
Query OK, 0 rows affected (0.02 sec)
```

Create timetable table:

Command: `CREATE TABLE timetable(timetable_id INT PRIMARY KEY, day_of_week VARCHAR(10), start_time TIME, end_time TIME, room_number VARCHAR(10));`



```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE timetable(
  -> timetable_id INT PRIMARY KEY,
  -> day_of_week VARCHAR(10),
  -> start_time TIME,
  -> end_time TIME,
  -> room_number VARCHAR(10)
  -> );
Query OK, 0 rows affected (0.02 sec)

mysql> _
```

Create assessment table;

Command: CREATE TABLE ASSESSMENT(ASSESSMENT_ID INT PRIMARY KEY, ASSESSMENT_TYPE VARCHAR(10), DATE DATE, MARKS INT);

```
MySQL 8.0 Command Line Client
mysql> create table assessment(assessment_id INT PRIMARY KEY, asses
sment_type VARCHAR(10),date DATE, marks INT);
Query OK, 0 rows affected (0.03 sec)

mysql>
```

Create Marks table:

Command: CREATE TABLE marks(marks_id INT PRIMARY KEY, student_id INT, course_id INT, marks_obtained INT);

```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE marks(marks_id INT PRIMARY KEY, student_id INT,
course_id INT, marks_obtained INT);
Query OK, 0 rows affected (0.03 sec)

mysql> _
```

Show all the tables:

Command: SHOW TABLES;

```
MySQL 8.0 Command Line Client
mysql> SHOW TABLES;
+-----+
| Tables_in_college_academic_system |
+-----+
| assessment                         |
| attendance                         |
| course                             |
| department                         |
| enrollment                         |
| faculty                           |
| student                           |
| timetable                         |
+-----+
8 rows in set (0.00 sec)

mysql> _
```

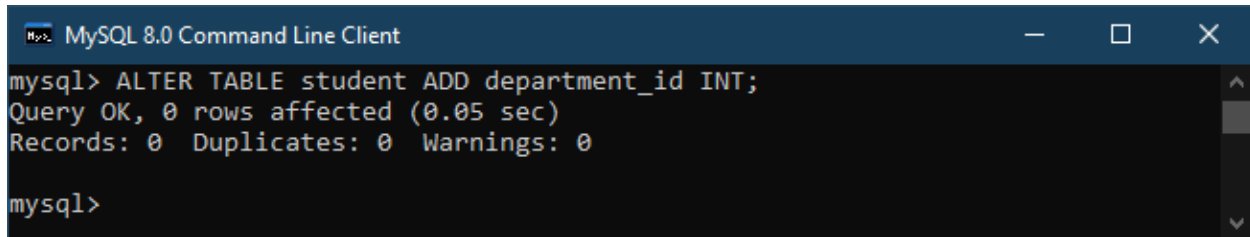
Add FOREIGN KEYS (RELATION)

Student has relation with department

- Student is dependent to department.
- Department is independent entity.
- Add primary key of department to student table as a foreign key.

Add attribute department_id in student table.

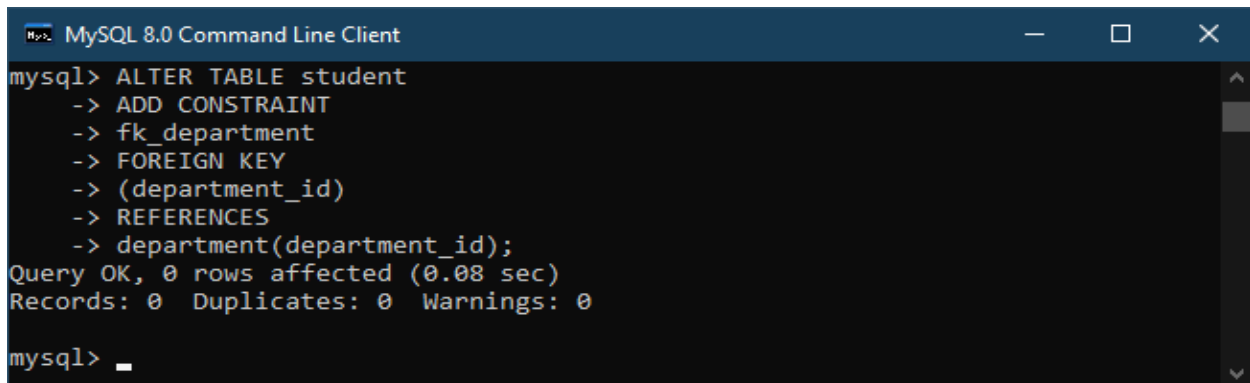
Command: ALTER TABLE student ADD department_id INT;



```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE student ADD department_id INT;
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
mysql>
```

Add reference to the parent table(department)

Command: ALTER TABLE student ADD CONSTRAINT fk_department FOREIGN KEY (department_id) REFERENCES department(department_id);



```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE student
-> ADD CONSTRAINT
-> fk_department
-> FOREIGN KEY
-> (department_id)
-> REFERENCES
-> department(department_id);
Query OK, 0 rows affected (0.08 sec)
Records: 0 Duplicates: 0 Warnings: 0
mysql> _
```

Course has Relation with department:

- Course is dependent to department.
- Department is an independent entity.
- Add primary key of department to course table as a foreign key.

Add attribute name department_id in the course table.

Command: ALTER TABLE course ADD department_id INT;

```
MySQL 8.0 Command Line Client

mysql> ALTER TABLE course ADD department_id INT;
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> _
```

Add reference to the parent table (department) as a foreign key.

Command: ALTER TABLE course ADD CONSTRAINT fk_department_id
FOREIGN KEY (department_id) REFERENCES
department(department_id);

```
MySQL 8.0 Command Line Client

mysql> ALTER TABLE course
-> ADD CONSTRAINT
-> fk_department_id
-> FOREIGN KEY
-> (department_id)
-> REFERENCES
-> department(department_id);
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
```

Faculty has relation with department

- Faculty is dependent to department.
- Department is an independent entity.

Add attribute named department in the faculty table.

Command: ALTER TABLE faculty ADD department_id INT;


```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE faculty
    -> add department_id INT;
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> _
```

Reference this attribute to the parent table (department) as foreign key.

Command: ALTER TABLE faculty ADD CONSTRAINT
fk_department_id_faculty FOREIGN KEY (department_id) REFERENCES
department(department_id);

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE faculty
    -> ADD CONSTRAINT
    -> fk_department_id_faculty
    -> FOREIGN KEY
    -> (department_id)
    -> REFERENCES
    -> department(department_id);
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> _
```

Enrollment has relation with student and course

- Enrollment(dependent)
- Student, course(independent)

Add attribute student_id and course_id to Enrollment table.

Command: ALTER TABLE enrollment ADD student_id INT;

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE enrollment
-> ADD
-> student_id INT;
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
```

Command: ALTER TABLE enrollment ADD course_id INT;

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE enrollment
-> ADD
-> course_id INT;
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> _
```

References these attributes as foreign key to parent table(student and course)

Command: ALTER TABLE enrollment ADD CONSTRAINT
fk_student_id_enrollment FOREIGN KEY (student_id) REFERENCES
student(student_id);

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE enrollment
-> ADD CONSTRAINT
-> fk_student_id_enrollment
-> FOREIGN KEY
-> (student_id)
-> REFERENCES
-> student(student_id);
Query OK, 0 rows affected (0.08 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

Command: ALTER TABLE enrollment ADD CONSTRAINT
fk_course_id_enrollment FOREIGN KEY (course_id) REFERENCES
course(course_id);

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE enrollment
-> ADD CONSTRAINT
-> fk_course_id_enrollment
-> FOREIGN KEY
-> (course_id)
-> REFERENCES
-> course(course_id);
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

Assessment relates to faculty and course

- Assessment(dependent)
- Faculty(independent)
- Course(independent)

Add faculty_id to assessment_table;

Command: ALTER TABLE assessment ADD faculty_id INT;

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE assessment
-> ADD
-> faculty_id INT;
Query OK, 0 rows affected (0.07 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
```

Add course_id to assessment table.

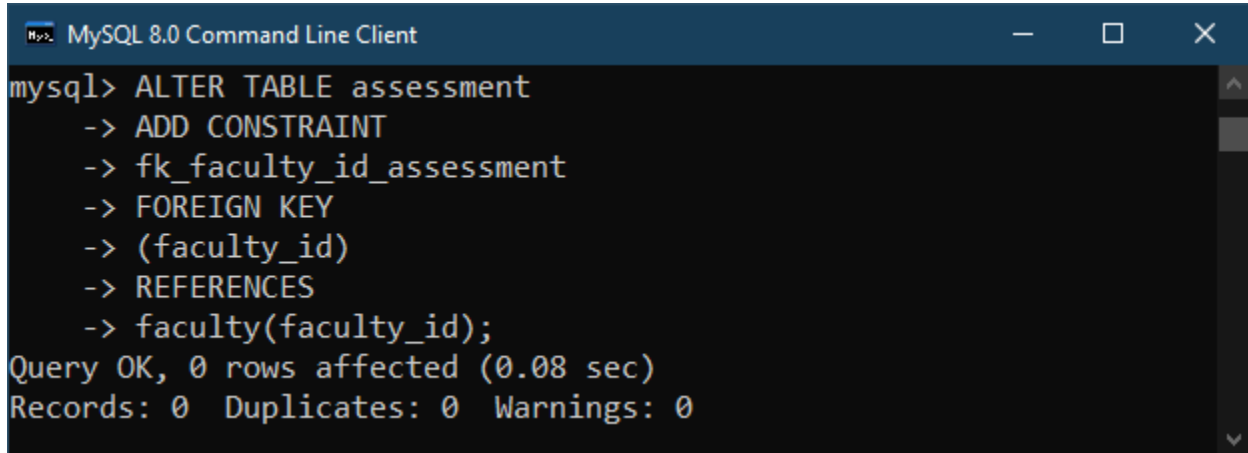
Command: ALTER TABLE assessment ADD course_id INT;

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE assessment
-> ADD
-> course_id INT;
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> _
```

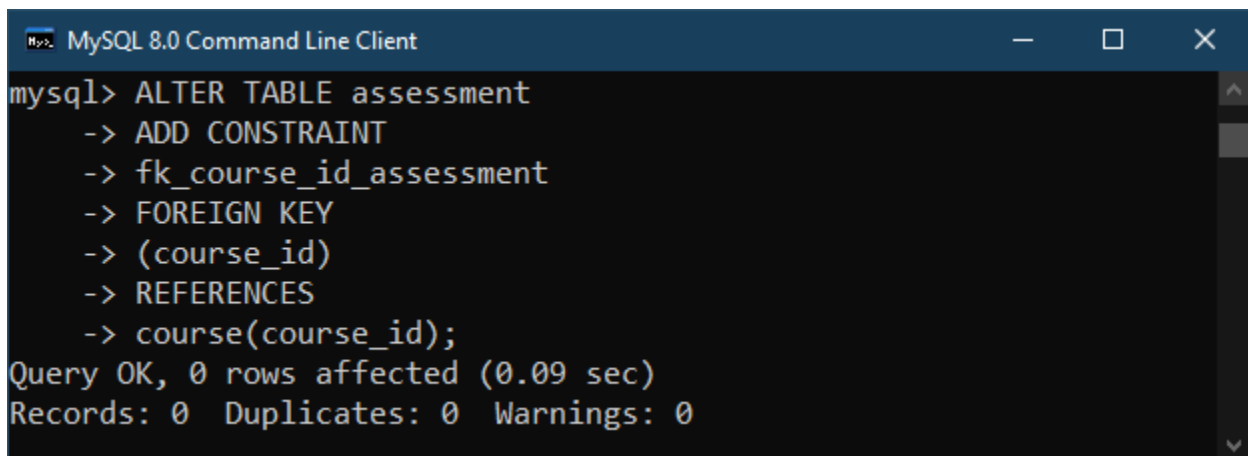
Add references as foreign key

Command: ALTER TABLE ASSESSMENT ADD CONSTRAINT
fk_faculty_id_assessment FOREIGN KEY (FACULTY_ID) REFERENCES
FACULTY(FACULTY_ID);



```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE assessment
-> ADD CONSTRAINT
-> fk_faculty_id_assessment
-> FOREIGN KEY
-> (faculty_id)
-> REFERENCES
-> faculty(faculty_id);
Query OK, 0 rows affected (0.08 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

Command: ALTER TABLE ASSESSMENT ADD CONSTRAINT
fk_COURSE_id_assessment FOREIGN KEY (COURSE_ID) REFERENCES
COURSE(COURSE_ID);



```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE assessment
-> ADD CONSTRAINT
-> fk_course_id_assessment
-> FOREIGN KEY
-> (course_id)
-> REFERENCES
-> course(course_id);
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

Attendance has relation with student and related course:

As student and course are independent entities and attendance is dependent to them because student and course can exist without attendance but attendance can't do so.

Add primary of course and student to the attendance table to make relationship.

Add the attributes course_id and student_id to the attendance table.

Command: ALTER TABLE attendance ADD student_id INT;

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE attendance ADD student_id INT;
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> _
```

Add the course_id :

Command: ALTER TABLE attendance ADD course_id INT;

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE attendance ADD course_id INT;
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> _
```

Add Reference the parent tables (student, course) to the attendance table.

Command : ALTER TABLE attendance ADD CONSTRAINT
fk_student_id_attendance FOREIGN KEY (student_id) REFERENCES
student(student_id);

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE attendance ADD CONSTRAINT fk_student_id_attendan
ce FOREIGN KEY (student_id) REFERENCES student(student_id);
Query OK, 0 rows affected (0.12 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> _
```

Command: ALTER TABLE attendance ADD CONSTRAINT
fk_course_id_attendance FOREIGN KEY (course_id) REFERENCES
course(course_id);

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE attendance ADD CONSTRAINT fk_course_id_attendance FOREIGN KEY (course_id) REFERENCES course(course_id);
Query OK, 0 rows affected (0.10 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> _
```

Add and reference the course_id and student_id in assessment table.

Command: ALTER TABLE assessment ADD CONSTRAINT
fk_student_assessment FOREIGN KEY (student_id) REFERENCES
student(student_id);

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE assessment ADD CONSTRAINT fk_student_assessment FOREIGN KEY (student_id) REFERENCES student(student_id);
Query OK, 0 rows affected (0.10 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> _
```

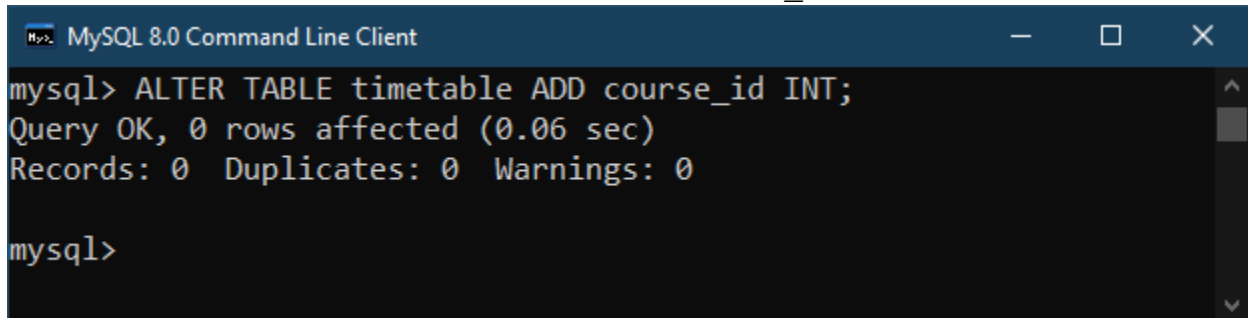
Command: ALTER TABLE assessment ADD CONSTRAINT
fk_course_assessment FOREIGN KEY (course_id) REFERENCES
course(course_id);

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE assessment ADD CONSTRAINT fk_course_assessment FOREIGN KEY (course_id) REFERENCES course(course_id);
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> _
```

Add attribute `course_id` and `faculty_id` in `timetable` table because it helps us to find out `max_teaching` load.

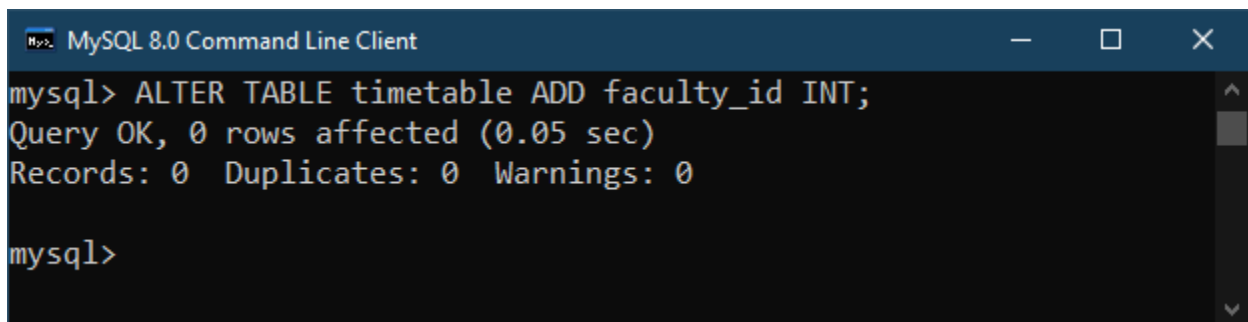
Command: `ALTER TABLE timetable ADD course_id INT;`



```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE timetable ADD course_id INT;
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
```

Command: `ALTER TABLE timetable ADD faculty_id INT;`

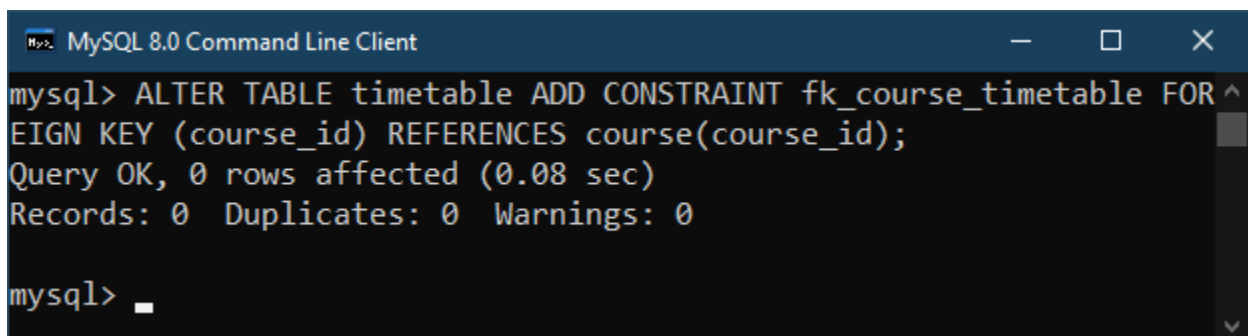


```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE timetable ADD faculty_id INT;
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
```

Add references:

Command: `ALTER TABLE timetable ADD CONSTRAINT fk_course_timetable FOREIGN KEY (course_id) REFERENCES course(course_id);`



```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE timetable ADD CONSTRAINT fk_course_timetable FOR
EIGN KEY (course_id) REFERENCES course(course_id);
Query OK, 0 rows affected (0.08 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> _
```

Command: `ALTER TABLE timetable ADD CONSTRAINT fk_course_timetable FOREIGN KEY (course_id) REFERENCES course(course_id);`

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE timetable ADD CONSTRAINT fk_faculty_timetable FOREIGN KEY (faculty_id) REFERENCES faculty(faculty_id);
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
```

Reference the marks table:

Command: ALTER TABLE marks ADD CONSTRAINT fk_course_marks FOREIGN KEY (course_id) REFERENCES course(course_id);

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE marks ADD CONSTRAINT fk_course_marks FOREIGN KEY (course_id) REFERENCES course(course_id);
Query OK, 0 rows affected (0.12 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
```

Comamnd : ALTER TABLE marks ADD CONSTRAINT fk_student_marks FOREIGN KEY (student_id) REFERENCES student(student_id);

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE marks ADD CONSTRAINT fk_student_marks FOREIGN KEY (student_id) REFERENCES student(student_id);
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
```


Show the description of tables.

Command: DESC table_name;

Description of student Table:

Command: DESC student;

```
MySQL 8.0 Command Line Client
mysql> DESC student;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| student_id     | int           | NO   | PRI | NULL    |       |
| student_name   | varchar(100)  | YES  |     | NULL    |       |
| student_email  | varchar(100)  | YES  |     | NULL    |       |
| date_of_birth  | date          | YES  |     | NULL    |       |
| department_id  | int           | YES  | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
```

Description of faculty table:

Command: DESC faculty;

```
MySQL 8.0 Command Line Client
mysql> DESC faculty;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| faculty_id     | int           | NO   | PRI | NULL    |       |
| faculty_name   | varchar(100)  | YES  |     | NULL    |       |
| max_teaching_load | int          | YES  |     | NULL    |       |
| department_id  | int           | YES  | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> _
```

Description of department table:

Command: DESC department;

```
MySQL 8.0 Command Line Client
mysql> DESC department;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| department_id  | int           | NO   | PRI | NULL    |       |
| department_name | varchar(100)  | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

Description of course table:

Command: DESC course;

```
MySQL 8.0 Command Line Client
mysql> desc course;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| course_id      | int           | NO   | PRI | NULL    |       |
| course_name    | varchar(100)  | YES  |     | NULL    |       |
| course_code    | varchar(100)  | YES  |     | NULL    |       |
| department_id  | int           | YES  | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Description of enrollment table:

Command: DESC enrollment;

```
MySQL 8.0 Command Line Client
mysql> DESC enrollment;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| enrollment_id  | int           | NO   | PRI | NULL    |       |
| semester       | varchar(20)    | YES  |     | NULL    |       |
| year           | int           | YES  |     | NULL    |       |
| grade          | varchar(2)     | YES  |     | NULL    |       |
| student_id     | int           | YES  | MUL | NULL    |       |
| course_id      | int           | YES  | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

Description of attendance table:

Command: DESC attendance;

```
MySQL 8.0 Command Line Client
mysql> DESC attendance;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| attendance_id  | int           | NO   | PRI | NULL    |       |
| date           | date          | YES  |     | NULL    |       |
| status         | varchar(10)    | YES  |     | NULL    |       |
| student_id     | int           | YES  | MUL | NULL    |       |
| course_id      | int           | YES  | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Description of assessment table:

Command: DESC assessment;

```
MySQL 8.0 Command Line Client
mysql> DESC assessment;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| assessment_id  | int           | NO   | PRI | NULL    |       |
| assessment_type | varchar(10)   | YES  |     | NULL    |       |
| date           | date          | YES  |     | NULL    |       |
| marks          | int           | YES  |     | NULL    |       |
| student_id     | int           | YES  | MUL | NULL    |       |
| course_id      | int           | YES  | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

Description of marks table:

Command: DESC marks;

```
MySQL 8.0 Command Line Client
mysql> DESC marks;
+-----+-----+-----+-----+-----+-----+
| Field          | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| marks_id       | int  | NO   | PRI | NULL    |       |
| student_id     | int  | YES  | MUL | NULL    |       |
| course_id      | int  | YES  | MUL | NULL    |       |
| marks_obtained | int  | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Description of timetable table:

Command: DESC timetable;

```
MySQL 8.0 Command Line Client
mysql> DESC timetable;
```

Field	Type	Null	Key	Default	Extra
timetable_id	int	NO	PRI	NULL	
day_of_week	varchar(10)	YES		NULL	
start_time	time	YES		NULL	
end_time	time	YES		NULL	
room_number	varchar(10)	YES		NULL	
course_id	int	YES	MUL	NULL	
faculty_id	int	YES	MUL	NULL	

INSERT RECORDS:

Single record entry:

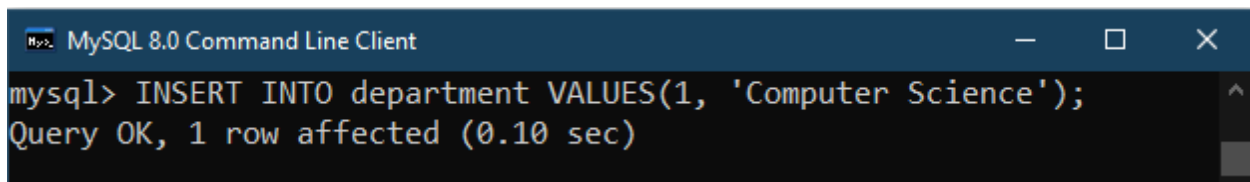
Command: `INSERT INTO table_name VALUES(value1, value2, ....);`

Multi-records entry:

Command: `INSERT INTO table_name Values(value1, value2, ...), (value1, value2, ...), (value1, value2, ...), ..... (value1, value2, ...);`

Insert records in department table:

`INSERT INTO department VALUES(1, 'Computer Science');`

A screenshot of the MySQL 8.0 Command Line Client window. The title bar reads "MySQL 8.0 Command Line Client". The command prompt shows the command `mysql> INSERT INTO department VALUES(1, 'Computer Science');` being executed. The output below the command is `Query OK, 1 row affected (0.10 sec)`. The window has standard Windows window controls (minimize, maximize, close) in the top right corner.

Command: `INSERT INTO department VALUES`
 `-> (2, 'chemistry'),`
 `-> (3, 'Physics'),`
 `-> (4, 'Mathematics'),`
 `-> (5, 'Zoology'),`
 `-> (6, 'Urdu'),`
 `-> (7, 'English'),`
 `-> (8, 'Economics'),`
 `-> (9, 'Islamiyat'),`
 `-> (10, 'Political Science');`

```
MySQL 8.0 Command Line Client
mysql> INSERT INTO department VALUES
-> (2, 'chemistry'),
-> (3, 'Physics'),
-> (4, 'Mathematics'),
-> (5, 'Zoology'),
-> (6, 'Urdu'),
-> (7, 'English'),
-> (8, 'Economics'),
-> (9, 'Islamiyat'),
-> (10, 'Political Science');
Query OK, 9 rows affected (0.00 sec)
Records: 9 Duplicates: 0 Warnings: 0

mysql> _
```

INSERT RECORDS IN student table

INSERT INTO student VALUES

```
-> (110837, 'Muhammad Muzzamal', 'muzzamal@gmail.com', '2005-05-03', 1),
-> (110855, 'Imran Ali', 'imranali@gamil.com', '2005-06-028', 1),
-> (110812, 'Muhammad Waseem', 'waseem@gmail.com', '2005-07-08', 1);
```

```
MySQL 8.0 Command Line Client
mysql> INSERT INTO student VALUES
-> (110837, 'Muhammad Muzzamal', 'muzzamal@gmail.com', '2005-05-03', 1),
-> (110855, 'Imran Ali', 'imranali@gamil.com', '2005-06-028', 1),
-> (110812, 'Muhammad Waseem', 'waseem@gmail.com', '2005-07-08', 1);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> _
```

Command:

INSERT INTO course VALUES

```
-> (1, 'DBMS', 'cc-111', 1),
-> (2, 'DSA', 'cc-112', 1),
-> (3, 'Software Engineering', 'cc-113', 1),
-> (4, 'Modeling and simulation', 'cc-114', 1);
```

```
MySQL 8.0 Command Line Client
mysql> INSERT INTO course VALUES
-> (1, 'DBMS', 'cc-111', 1),
-> (2, 'DSA', 'cc-112', 1),
-> (3, 'Software Engineering', 'cc-113', 1),
-> (4, 'Modeling and simulation', 'cc-114', 1);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
```

INSERT INTO faculty VALUES

```
-> (1, 'Professor Hassan Raza', 4, 1),
-> (2, 'Professor Ali Waqas', 2, 1),
-> (3, 'Madam Sehrish Khan', 4, 1),
-> (4, 'Professor Kamran', 4, 1),
-> (5, 'Professor Gufar', 4, 1);
```

```
MySQL 8.0 Command Line Client
mysql> INSERT INTO faculty VALUES
-> (1, 'Professor Hassan Raza', 4, 1),
-> (2, 'Professor Ali Waqas', 2, 1),
-> (3, 'Madam Sehrish Khan', 4, 1),
-> (4, 'Professor Kamran', 4, 1),
-> (5, 'Professor Gufar', 4, 1);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql>
```

INSERT INTO enrollment VALUES

```
-> (1111, 4, 2025, 'A', 110837, 1),
-> (1112, 4, 2025, 'A', 110837, 1),
-> (1113, 3, 2024, 'B', 110837, 1),
-> (1114, 4, 2025, 'A', 110855, 1);
```



```
MySQL 8.0 Command Line Client
mysql> INSERT INTO enrollment VALUES
-> (1111, 4, 2025, 'A', 110837, 1),
-> (1112, 4, 2025, 'A', 110837, 1),
-> (1113, 3, 2024, 'B', 110837, 1),
-> (1114, 4, 2025, 'A', 110855, 1);
Query OK, 4 rows affected (0.02 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
```

INSERT INTO ATTENDANCE VALUES

```
-> (1114, '2025-05-05', 'Present', 110837, 1),
-> (1115, '2025-05-06', 'Present', 110837, 1),
-> (1116, '2025-05-06', 'Absent', 110837, 1),
-> (1117, '2025-05-07', 'Leave', 110837, 1);
```

```
MySQL 8.0 Command Line Client
mysql> INSERT INTO ATTENDANCE VALUES
-> (1114, '2025-05-05', 'Present', 110837, 1),
-> (1115, '2025-05-06', 'Present', 110837, 1),
-> (1116, '2025-05-06', 'Absent', 110837, 1),
-> (1117, '2025-05-07', 'Leave', 110837, 1);
Query OK, 4 rows affected (0.00 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
```

INSERT INTO assessment VALUES

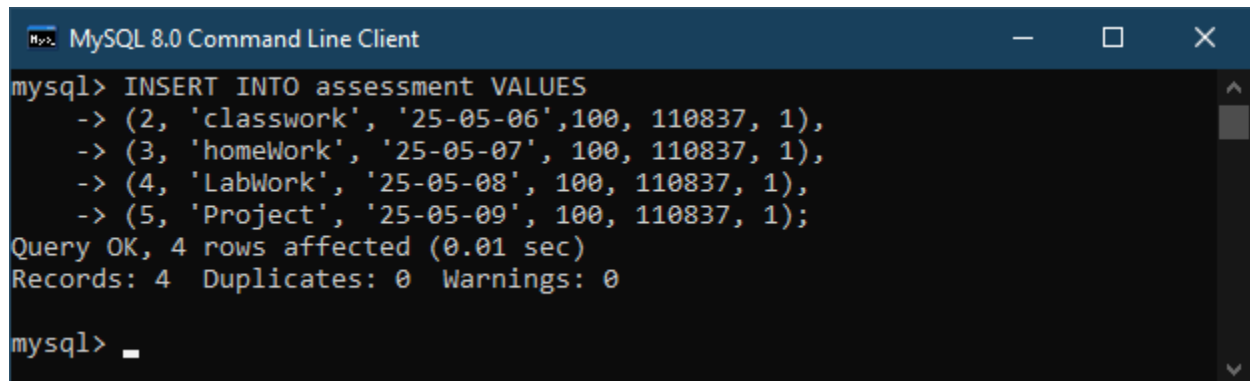
```
-> (1, 'Lab work', '25-05-05', 100, 110837, 1);
```

```
MySQL 8.0 Command Line Client
mysql> INSERT INTO assessment VALUES
-> (1, 'Lab work', '25-05-05', 100, 110837, 1);
Query OK, 1 row affected (0.01 sec)

mysql>
```

```
INSERT INTO assessment VALUES
```

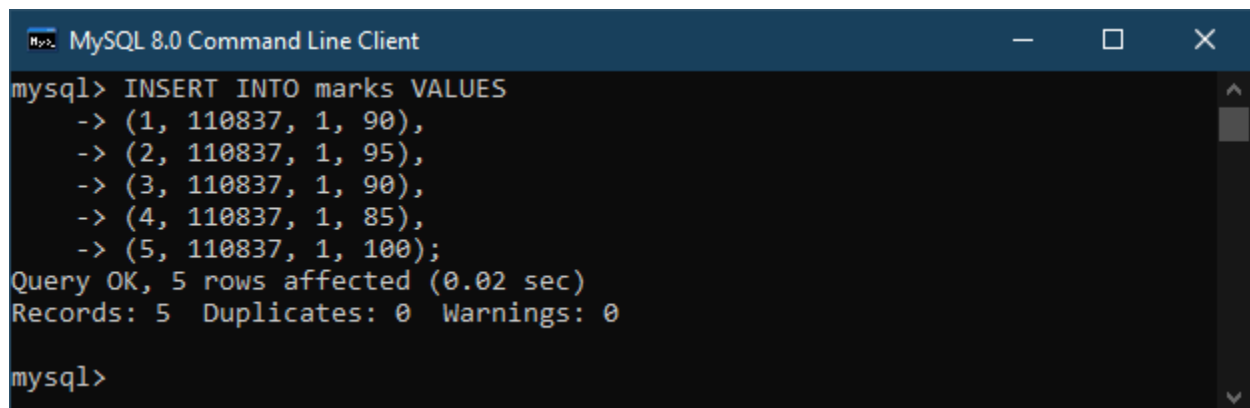
```
-> (2, 'classwork', '25-05-06',100, 110837, 1),  
-> (3, 'homeWork', '25-05-07', 100, 110837, 1),  
-> (4, 'LabWork', '25-05-08', 100, 110837, 1),0  
-> (5, 'Project', '25-05-09', 100, 110837, 1);
```



```
MySQL 8.0 Command Line Client  
mysql> INSERT INTO assessment VALUES  
-> (2, 'classwork', '25-05-06',100, 110837, 1),  
-> (3, 'homeWork', '25-05-07', 100, 110837, 1),  
-> (4, 'LabWork', '25-05-08', 100, 110837, 1),  
-> (5, 'Project', '25-05-09', 100, 110837, 1);  
Query OK, 4 rows affected (0.01 sec)  
Records: 4  Duplicates: 0  Warnings: 0  
  
mysql> _
```

```
INSERT INTO marks VALUES
```

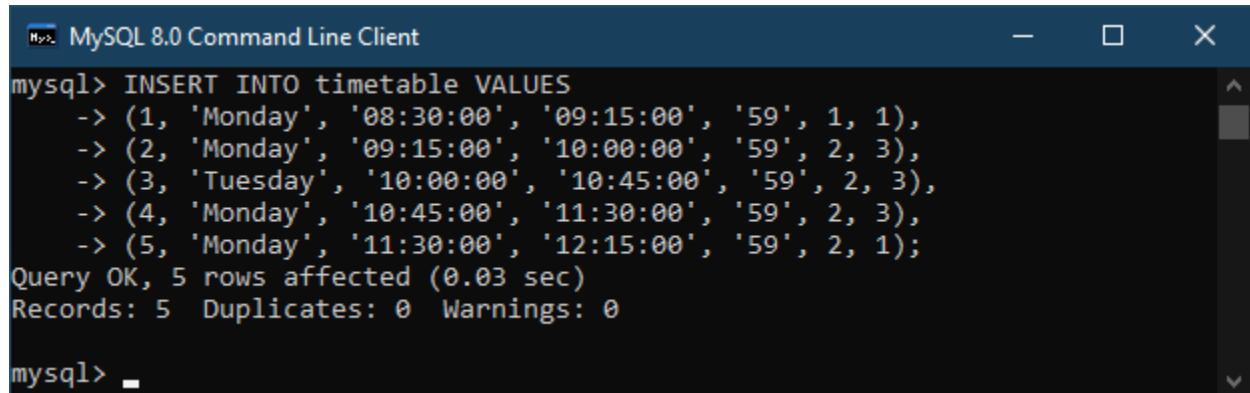
```
-> (1, 110837, 1, 90),  
-> (2, 110837, 1, 95),  
-> (3, 110837, 1, 90),  
-> (4, 110837, 1, 85),  
-> (5, 110837, 1, 100);
```



```
MySQL 8.0 Command Line Client  
mysql> INSERT INTO marks VALUES  
-> (1, 110837, 1, 90),  
-> (2, 110837, 1, 95),  
-> (3, 110837, 1, 90),  
-> (4, 110837, 1, 85),  
-> (5, 110837, 1, 100);  
Query OK, 5 rows affected (0.02 sec)  
Records: 5  Duplicates: 0  Warnings: 0  
  
mysql>
```

INSERT INTO timetable VALUES

```
-> (1, 'Monday', '08:30:00', '09:15:00', '59', 1, 1),
-> (2, 'Monday', '09:15:00', '10:00:00', '59', 2, 3),
-> (3, 'Tuesday', '10:00:00', '10:45:00', '59', 2, 3),
-> (4, 'Monday', '10:45:00', '11:30:00', '59', 2, 3),
-> (5, 'Monday', '11:30:00', '12:15:00', '59', 2, 1);
```



```
mysql> INSERT INTO timetable VALUES
-> (1, 'Monday', '08:30:00', '09:15:00', '59', 1, 1),
-> (2, 'Monday', '09:15:00', '10:00:00', '59', 2, 3),
-> (3, 'Tuesday', '10:00:00', '10:45:00', '59', 2, 3),
-> (4, 'Monday', '10:45:00', '11:30:00', '59', 2, 3),
-> (5, 'Monday', '11:30:00', '12:15:00', '59', 2, 1);
Query OK, 5 rows affected (0.03 sec)
Records: 5  Duplicates: 0  Warnings: 0

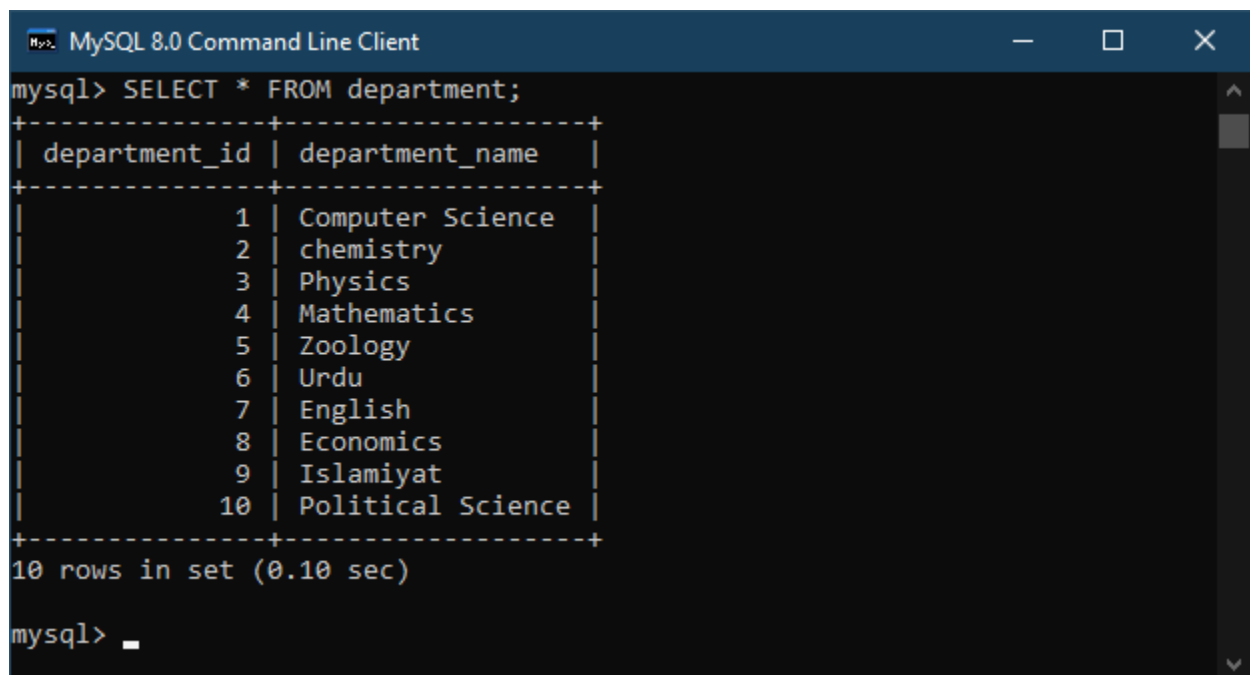
mysql>
```

SELECT STATEMENT:

Show the all records of a table:

Show the whole departments in a college:

Command: SELECT * FROM department;

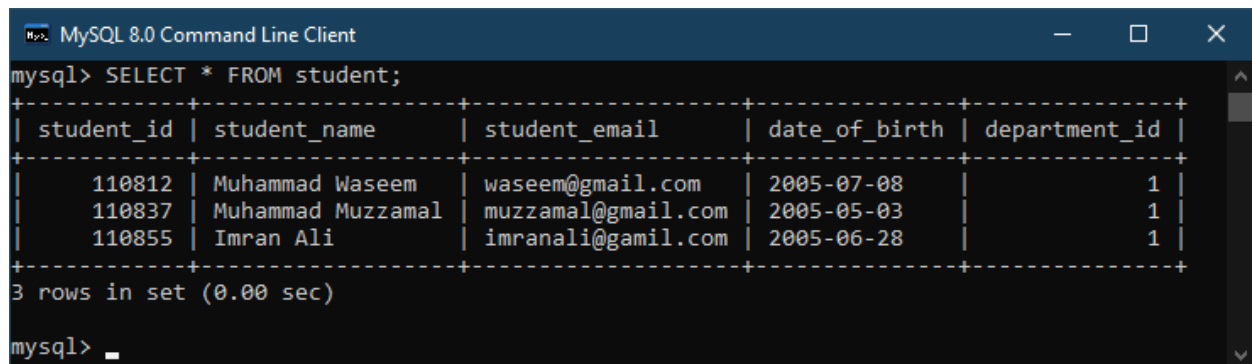


```
mysql> SELECT * FROM department;
+-----+-----+
| department_id | department_name |
+-----+-----+
| 1             | Computer Science |
| 2             | chemistry        |
| 3             | Physics          |
| 4             | Mathematics      |
| 5             | Zoology          |
| 6             | Urdu             |
| 7             | English          |
| 8             | Economics        |
| 9             | Islamiyat        |
| 10            | Political Science |
+-----+-----+
10 rows in set (0.10 sec)

mysql>
```

All records of Student in a college:

```
SELECT * FROM student;
```

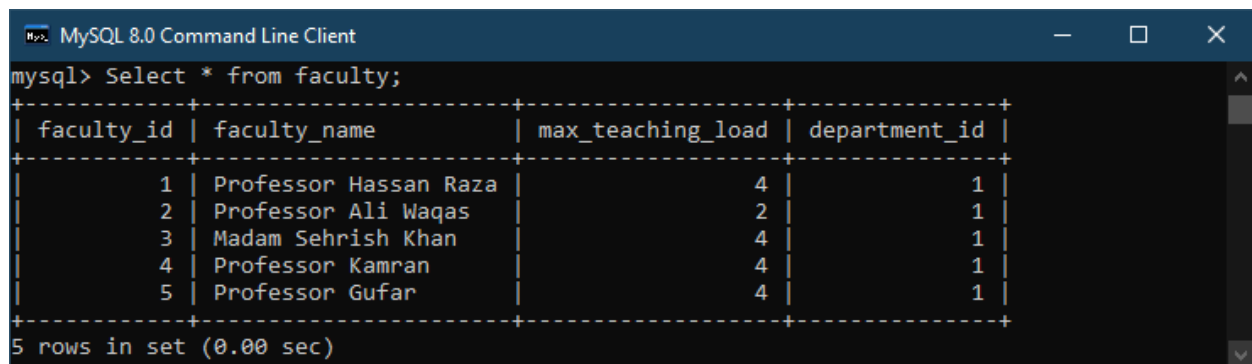


The screenshot shows the MySQL 8.0 Command Line Client window. The command prompt shows 'mysql> SELECT * FROM student;'. The output is a table with 5 columns: student_id, student_name, student_email, date_of_birth, and department_id. There are 3 rows of data. Below the table, it says '3 rows in set (0.00 sec)'. The prompt 'mysql>' is followed by a cursor.

student_id	student_name	student_email	date_of_birth	department_id
110812	Muhammad Waseem	waseem@gmail.com	2005-07-08	1
110837	Muhammad Muzzamal	muzzamal@gmail.com	2005-05-03	1
110855	Imran Ali	imranali@gamil.com	2005-06-28	1

All faculty in the college:

```
Select * from faculty;
```

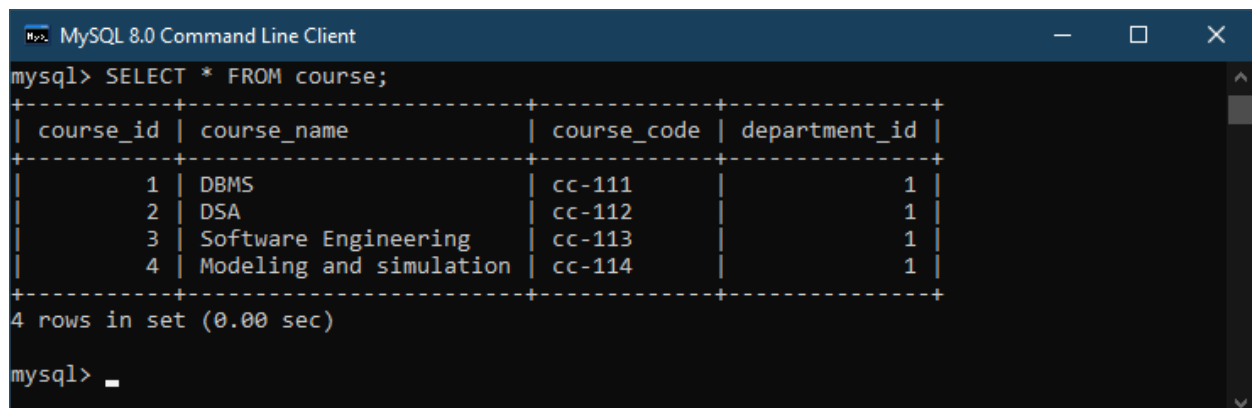


The screenshot shows the MySQL 8.0 Command Line Client window. The command prompt shows 'mysql> Select * from faculty;'. The output is a table with 4 columns: faculty_id, faculty_name, max_teaching_load, and department_id. There are 5 rows of data. Below the table, it says '5 rows in set (0.00 sec)'. The prompt 'mysql>' is followed by a cursor.

faculty_id	faculty_name	max_teaching_load	department_id
1	Professor Hassan Raza	4	1
2	Professor Ali Waqas	2	1
3	Madam Sehrish Khan	4	1
4	Professor Kamran	4	1
5	Professor Gufar	4	1

All records of course offered by college:

```
SELECT * FROM course;
```



The screenshot shows the MySQL 8.0 Command Line Client window. The command prompt shows 'mysql> SELECT * FROM course;'. The output is a table with 4 columns: course_id, course_name, course_code, and department_id. There are 4 rows of data. Below the table, it says '4 rows in set (0.00 sec)'. The prompt 'mysql>' is followed by a cursor.

course_id	course_name	course_code	department_id
1	DBMS	cc-111	1
2	DSA	cc-112	1
3	Software Engineering	cc-113	1
4	Modeling and simulation	cc-114	1

All recods in enrollment table:

```
SELECT * FROM enrollment;
```

```
MySQL 8.0 Command Line Client
mysql> SELECT * FROM enrollment;
+-----+-----+-----+-----+-----+-----+
| enrollment_id | semester | year | grade | student_id | course_id |
+-----+-----+-----+-----+-----+-----+
| 1111 | 4 | 2025 | A | 110837 | 1 |
| 1112 | 4 | 2025 | A | 110837 | 1 |
| 1113 | 3 | 2024 | B | 110837 | 1 |
| 1114 | 4 | 2025 | A | 110855 | 1 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)

mysql>
```

Show all the Attendance records:

```
SELECT * FROM attendance;
```

```
MySQL 8.0 Command Line Client
mysql> SELECT * FROM attendance;
+-----+-----+-----+-----+-----+-----+
| attendance_id | date       | status | student_id | course_id |
+-----+-----+-----+-----+-----+-----+
| 1114 | 2025-05-05 | Present | 110837 | 1 |
| 1115 | 2025-05-06 | Present | 110837 | 1 |
| 1116 | 2025-05-06 | Absent | 110837 | 1 |
| 1117 | 2025-05-07 | Leave | 110837 | 1 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)

mysql>
```

Show all Assessment Records:

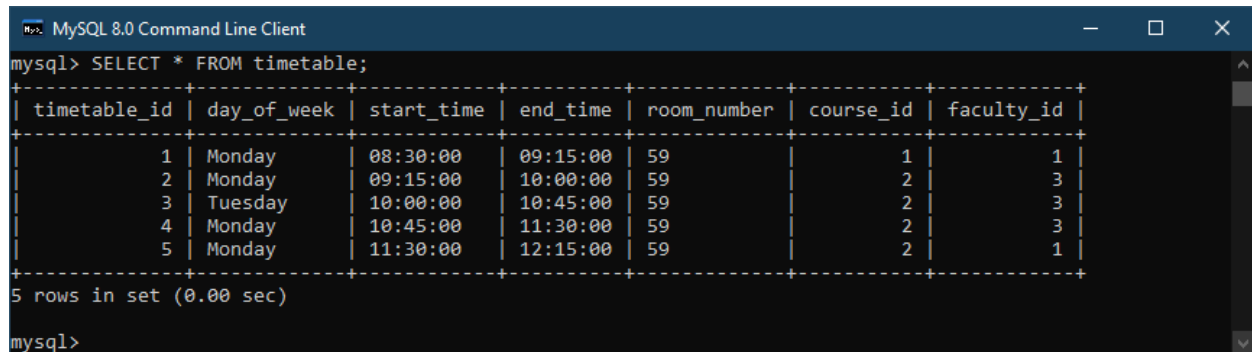
```
SELECT * FROM assessment;
```

```
MySQL 8.0 Command Line Client
mysql> SELECT * FROM assessment;
+-----+-----+-----+-----+-----+-----+
| assessment_id | assessment_type | date       | marks | student_id | course_id |
+-----+-----+-----+-----+-----+-----+
| 1 | Lab work | 2025-05-05 | 100 | 110837 | 1 |
| 2 | classwork | 2025-05-06 | 100 | 110837 | 1 |
| 3 | homework | 2025-05-07 | 100 | 110837 | 1 |
| 4 | LabWork | 2025-05-08 | 100 | 110837 | 1 |
| 5 | Project | 2025-05-09 | 100 | 110837 | 1 |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
```

Show Timetable Records:

`SELECT * FROM timetable;`



```
mysql> SELECT * FROM timetable;
```

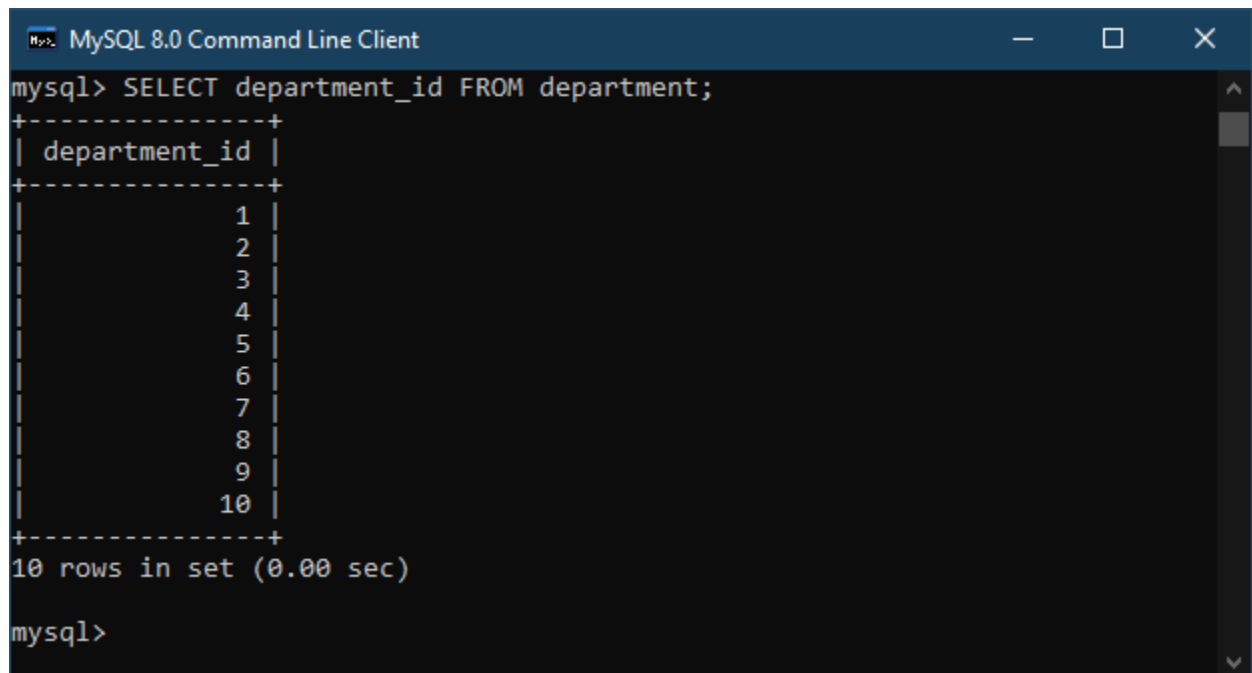
timetable_id	day_of_week	start_time	end_time	room_number	course_id	faculty_id
1	Monday	08:30:00	09:15:00	59	1	1
2	Monday	09:15:00	10:00:00	59	2	3
3	Tuesday	10:00:00	10:45:00	59	2	3
4	Monday	10:45:00	11:30:00	59	2	3
5	Monday	11:30:00	12:15:00	59	2	1

```
5 rows in set (0.00 sec)

mysql>
```

Show the specific column in a records;

Command: `SELECT department_id FROM department;`



```
mysql> SELECT department_id FROM department;
```

department_id
1
2
3
4
5
6
7
8
9
10

```
10 rows in set (0.00 sec)

mysql>
```

USE OF WHERE CLAUSE:

We want to show the student name whose student id is equal to 110837 so we use where clause.

Command: `SELECT student_name FROM student WHERE student_id=110837;`

```
MySQL 8.0 Command Line Client
mysql> SELECT student_name FROM student WHERE student_id=110837;
+-----+
| student_name |
+-----+
| Muhammad Muzzamal |
+-----+
1 row in set (0.00 sec)

mysql>
```

GROUP BY Clause:

Command: `SELECT * FROM department GROUP BY department_id;`

```
Select MySQL 8.0 Command Line Client
mysql> SELECT * FROM department GROUP BY department_id;
+-----+-----+
| department_id | department_name |
+-----+-----+
| 1 | Computer Science |
| 2 | chemistry |
| 3 | Physics |
| 4 | Mathematics |
| 5 | Zoology |
| 6 | Urdu |
| 7 | English |
| 8 | Economics |
| 9 | Islamiyat |
| 10 | Political Science |
+-----+-----+
10 rows in set (0.00 sec)

mysql>
```

HAVING CLAUSE:

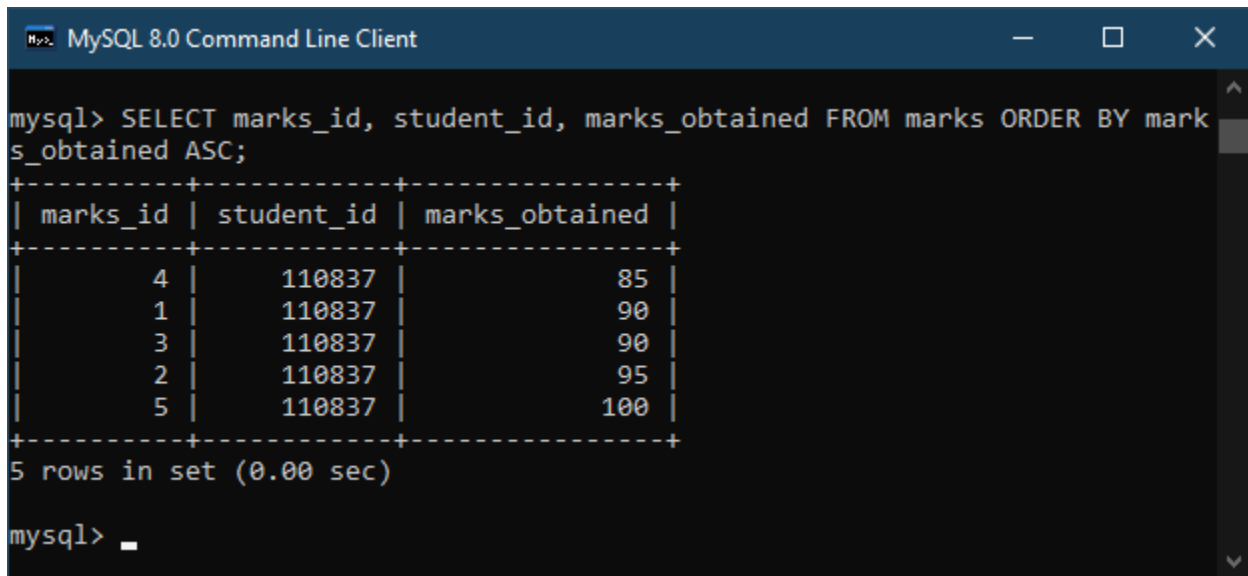
`SELECT * FROM marks GROUP BY marks_id HAVING (marks_obtained) > 90;`

```
MySQL 8.0 Command Line Client
mysql> SELECT * FROM marks GROUP BY marks_id HAVING (marks_obtained) > 90;
+-----+-----+-----+-----+
| marks_id | student_id | course_id | marks_obtained |
+-----+-----+-----+-----+
| 2 | 110837 | 1 | 95 |
| 5 | 110837 | 1 | 100 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

ORDERED BY CLAUSE:

```
SELECT marks_id, student_id, marks_obtained FROM marks ORDER BY marks_obtained ASC;
```

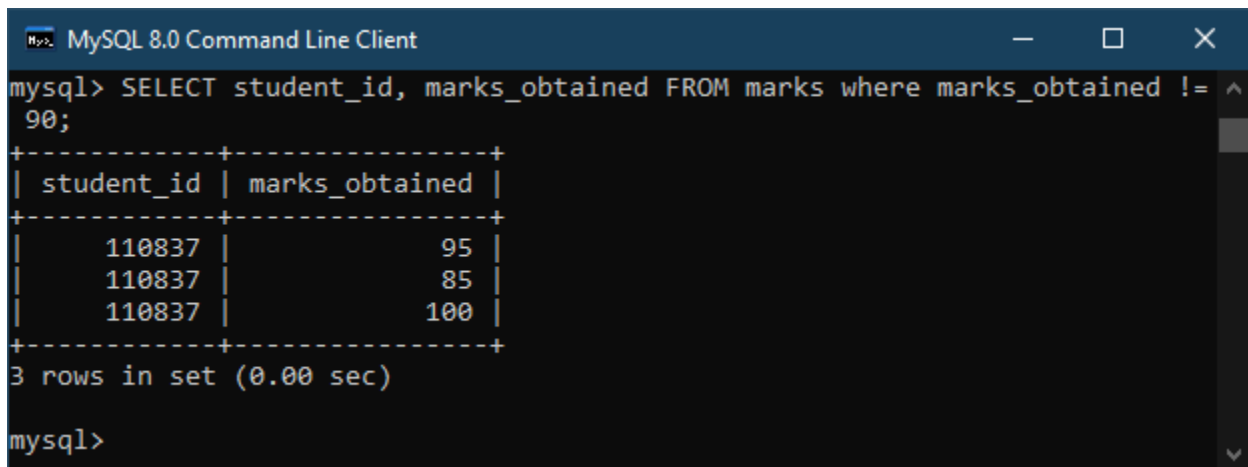


The screenshot shows a terminal window titled "MySQL 8.0 Command Line Client". The user has entered the command: `mysql> SELECT marks_id, student_id, marks_obtained FROM marks ORDER BY marks_obtained ASC;`. The output is a table with 5 rows, sorted by the `marks_obtained` column in ascending order. The table has columns `marks_id`, `student_id`, and `marks_obtained`. The rows are: (4, 110837, 85), (1, 110837, 90), (3, 110837, 90), (2, 110837, 95), and (5, 110837, 100). Below the table, it says "5 rows in set (0.00 sec)".

marks_id	student_id	marks_obtained
4	110837	85
1	110837	90
3	110837	90
2	110837	95
5	110837	100

Comparison Operators:

```
SELECT student_id, marks_obtained FROM marks where marks_obtained != 90;
```



The screenshot shows a terminal window titled "MySQL 8.0 Command Line Client". The user has entered the command: `mysql> SELECT student_id, marks_obtained FROM marks where marks_obtained != 90;`. The output is a table with 3 rows, where the `marks_obtained` column is not equal to 90. The table has columns `student_id` and `marks_obtained`. The rows are: (110837, 95), (110837, 85), and (110837, 100). Below the table, it says "3 rows in set (0.00 sec)".

student_id	marks_obtained
110837	95
110837	85
110837	100

```
SELECT student_id, marks_obtained FROM marks where marks_obtained < 90;
```



```
MySQL 8.0 Command Line Client

mysql> SELECT student_id, marks_obtained FROM marks where marks_obtained < 90;
+-----+-----+
| student_id | marks_obtained |
+-----+-----+
| 110837 | 85 |
+-----+-----+
1 row in set (0.00 sec)
```

SELECT student_id, marks_obtained FROM marks where marks_obtained != 90;

```
MySQL 8.0 Command Line Client

mysql> SELECT student_id, marks_obtained FROM marks where marks_obtained != 90;
+-----+-----+
| student_id | marks_obtained |
+-----+-----+
| 110837 | 95 |
| 110837 | 85 |
| 110837 | 100 |
+-----+-----+
3 rows in set (0.00 sec)
```

AGGREGATE FUNCTIONS:

SELECT MAX(marks_obtained) from marks;

```
MySQL 8.0 Command Line Client

mysql> SELECT MAX(marks_obtained) from marks;
+-----+
| MAX(marks_obtained) |
+-----+
| 100 |
+-----+
1 row in set (0.00 sec)

mysql> _
```

SELECT AVG(marks_obtained) from marks;

```
MySQL 8.0 Command Line Client
mysql> SELECT AVG(marks_obtained) from marks;
+-----+
| AVG(marks_obtained) |
+-----+
|          92.0000 |
+-----+
1 row in set (0.00 sec)

mysql>
```

SELECT MIN(marks_obtained) from marks;

```
MySQL 8.0 Command Line Client
mysql> SELECT MIN(marks_obtained) from marks;
+-----+
| MIN(marks_obtained) |
+-----+
|             85 |
+-----+
1 row in set (0.00 sec)

mysql> _
```

SELECT COUNT(marks_obtained) from marks WHERE marks_obtained > 90;

```
MySQL 8.0 Command Line Client
mysql> SELECT COUNT(marks_obtained) from marks WHERE marks_obtained > 90;
+-----+
| COUNT(marks_obtained) |
+-----+
|             2 |
+-----+
1 row in set (0.00 sec)

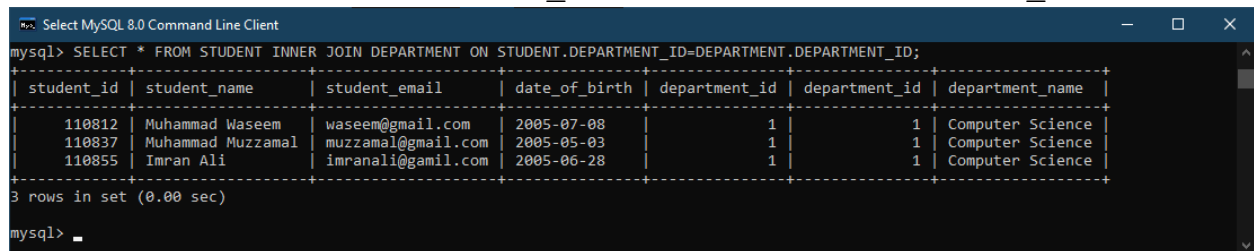
mysql>
```

CHAPTER # 8

Implementation of JOINS

Inner join :

```
SELECT * FROM STUDENT INNER JOIN  
DEPARTMENTSTUDENT.DEPARTMENT_ID=DEPARTMENT.DEPARTMENT_ID;
```



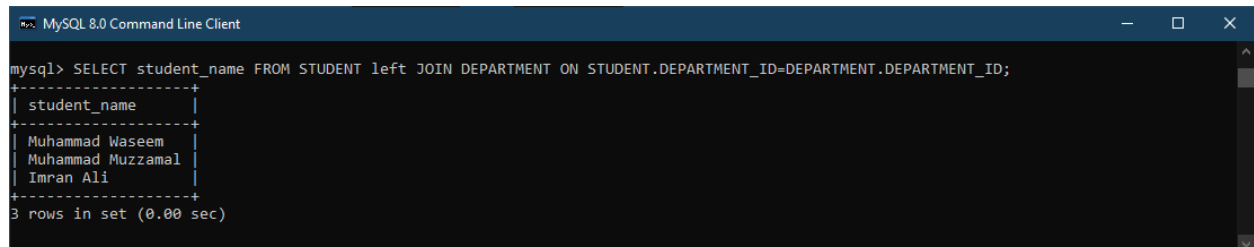
The screenshot shows a MySQL 8.0 Command Line Client window. The user has executed the query: `mysql> SELECT * FROM STUDENT INNER JOIN DEPARTMENT ON STUDENT.DEPARTMENT_ID=DEPARTMENT.DEPARTMENT_ID;`. The output is a table with 7 columns: `student_id`, `student_name`, `student_email`, `date_of_birth`, `department_id`, `department_id`, and `department_name`. There are 3 rows of data, all from the 'Computer Science' department. The status bar indicates '3 rows in set (0.00 sec)'.

student_id	student_name	student_email	date_of_birth	department_id	department_id	department_name
110812	Muhammad Waseem	waseem@gmail.com	2005-07-08	1	1	Computer Science
110837	Muhammad Muzzamal	muzzamal@gmail.com	2005-05-03	1	1	Computer Science
110855	Imran Ali	imranali@gmail.com	2005-06-28	1	1	Computer Science

Outer Join:

Left outer join:

```
SELECT student_name FROM STUDENT left JOIN DEPARTMENT ON  
STUDENT.DEPARTMENT_ID=DEPARTMENT.DEPARTMENT_ID;
```



The screenshot shows a MySQL 8.0 Command Line Client window. The user has executed the query: `mysql> SELECT student_name FROM STUDENT left JOIN DEPARTMENT ON STUDENT.DEPARTMENT_ID=DEPARTMENT.DEPARTMENT_ID;`. The output is a table with 2 columns: `student_name`. There are 3 rows of data, listing the names of the students from the 'Computer Science' department. The status bar indicates '3 rows in set (0.00 sec)'.

student_name
Muhammad Waseem
Muhammad Muzzamal
Imran Ali

Right Outer join:

```
SELECT student_name FROM STUDENT right JOIN DEPARTMENT ON  
STUDENT.DEPARTMENT_ID=DEPARTMENT.DEPARTMENT_ID;
```

```
MySQL 8.0 Command Line Client
mysql> SELECT student_name FROM STUDENT right JOIN DEPARTMENT ON STUDENT.DE
PARTMENT_ID=DEPARTMENT.DEPARTMENT_ID;
+-----+
| student_name |
+-----+
| Imran Ali    |
| Muhammad Muzzamal |
| Muhammad Waseem |
| NULL        |
| NULL        |
| NULL        |
| NULL        |
| NULL        |
| NULL        |
| NULL        |
| NULL        |
| NULL        |
+-----+
12 rows in set (0.00 sec)

mysql> _
```